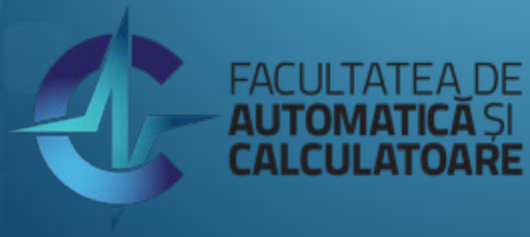



# Arhitecturi Paralele
# Abordări probleme paralele log(N)

**Radu-Ioan CIOBANU**
radu.ciobanu@upb.ro






# Sume prefix

# Aplicații folosind paralelismul de date
# Calculul sumelor prefix

Problemă:

Se dă tabloul **a[1:n]**, se cere **s[1:n]**, unde:

$$s[i] = \sum_{k=1}^{i} a[k]$$

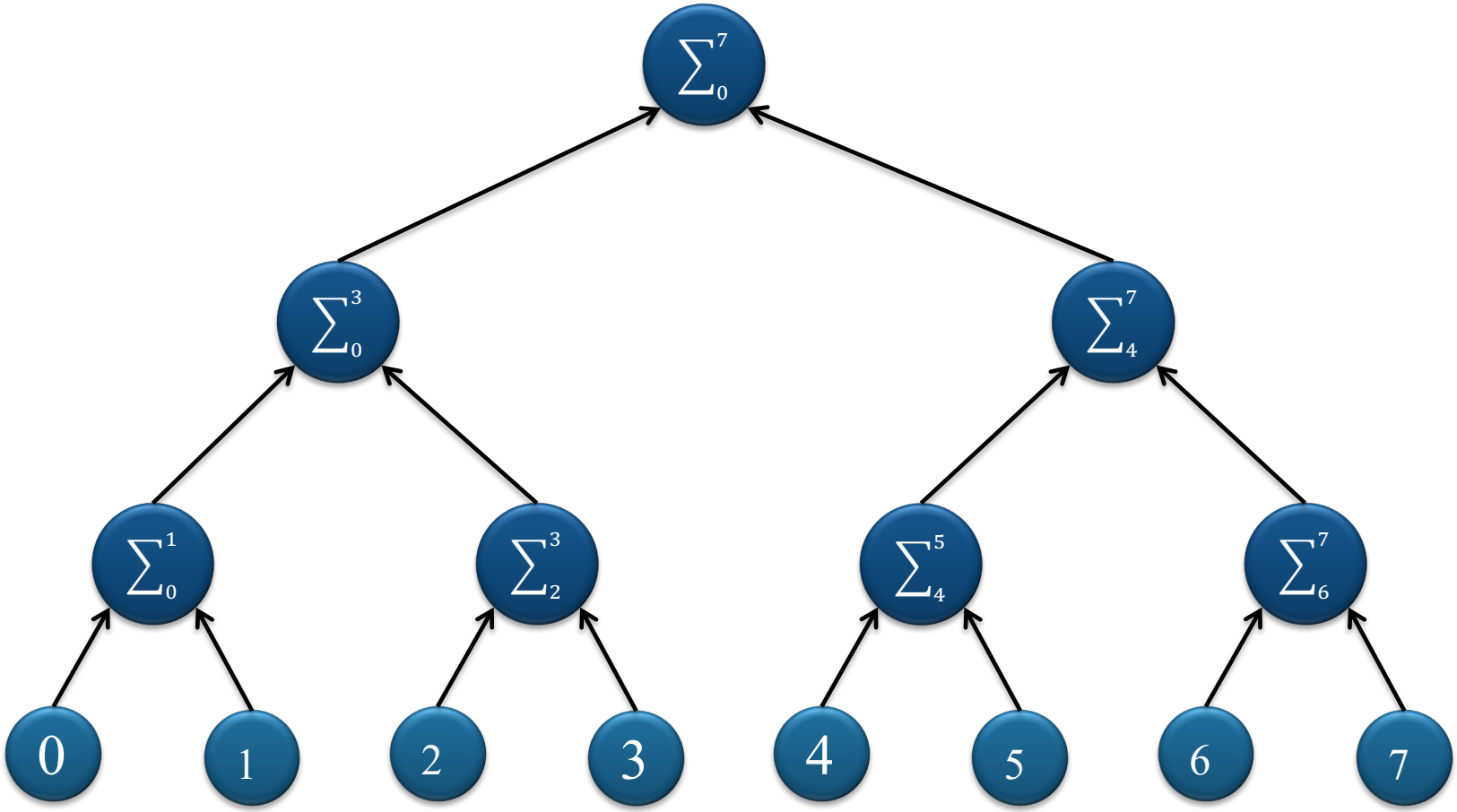
- Algoritm secvențial:

```
s[1] = a[1];
for [i = 2 to n]
    s[i] = a[i] + s[i-1];
```

- Algoritm paralel:

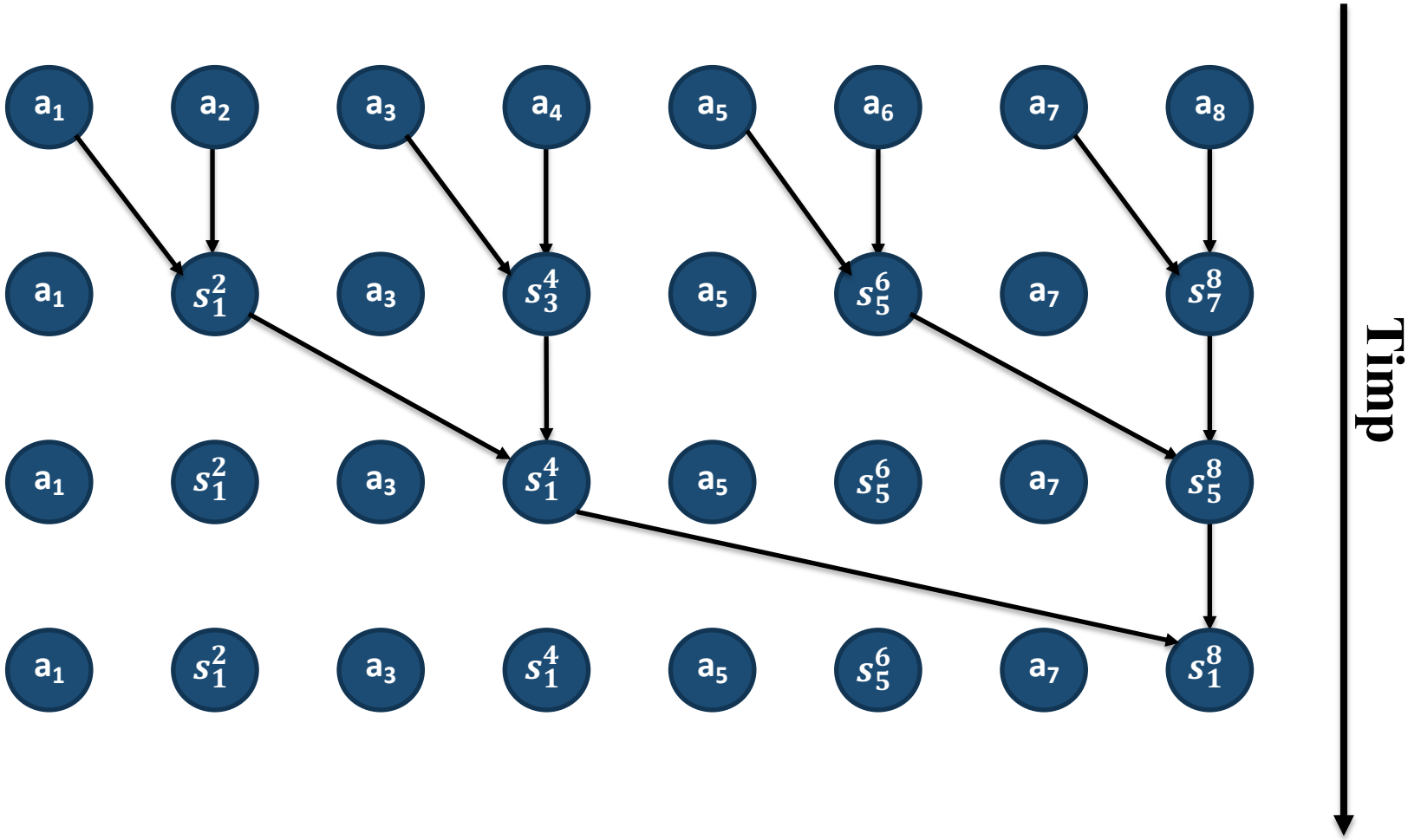
  - derivat din algoritmul ***sumei elementelor unui vector***

# Suma elementelor unui vector



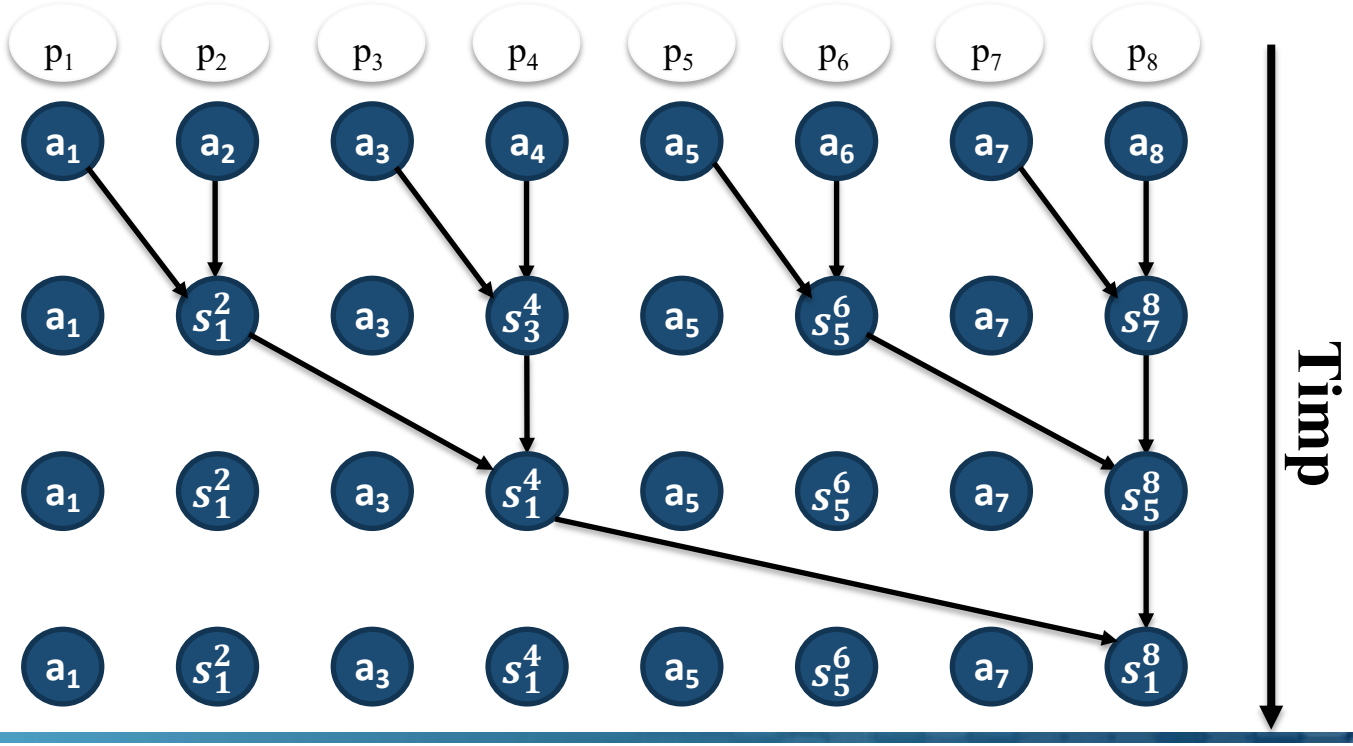

$\sup\left(\frac{n}{2}\right)$ procesoare, $\sup(\log_2 n)$ pași

# Suma elementelor unui vector

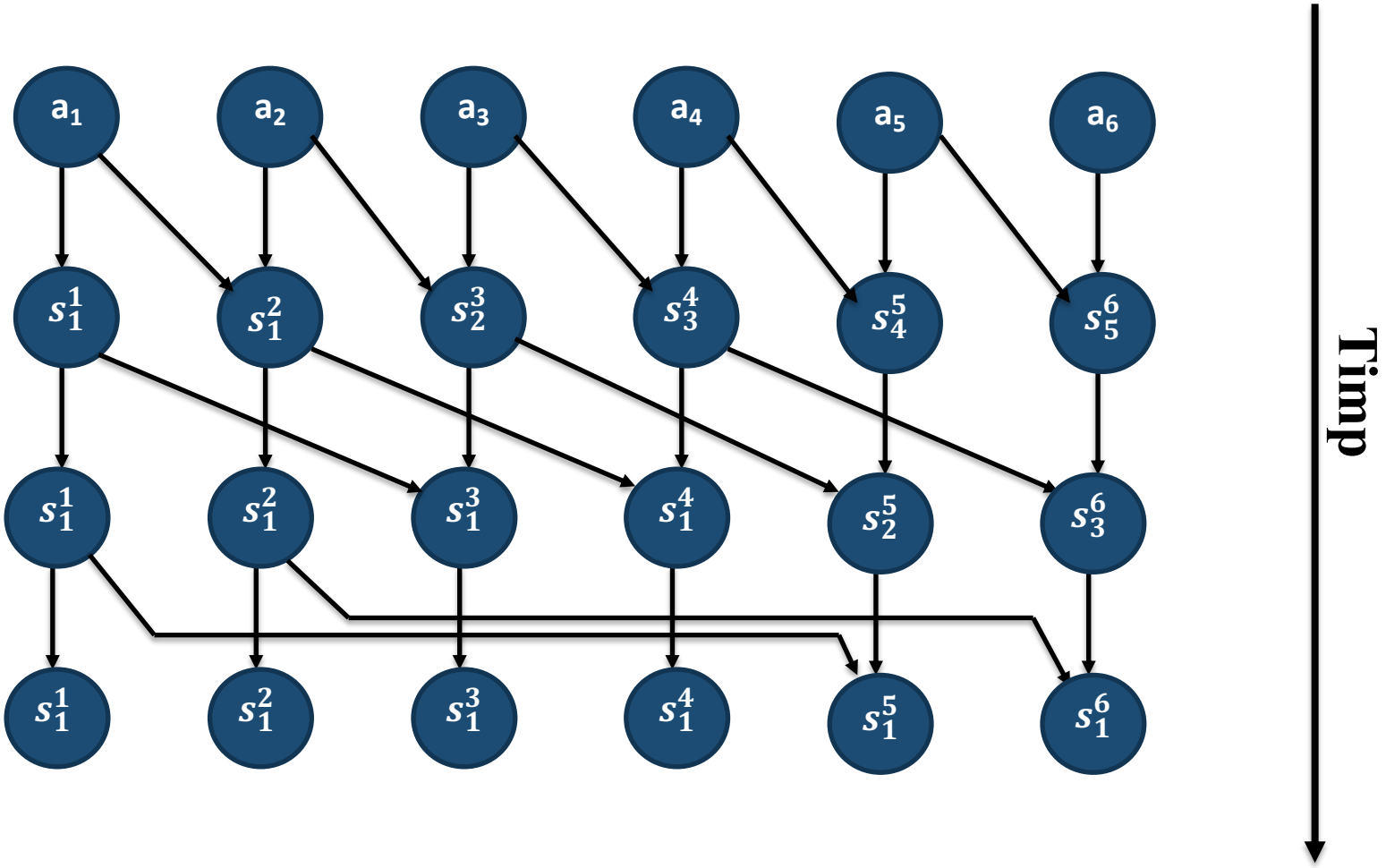


# Suma elementelor unui vector

```
int a[1:n];
process suma[k=1 to n] { // suma proceselor p din figura
      for [j = 1 to sup(log2 n)] {
            if (k mod 2^j == 0)
                  a[k] = a[k-2^(j-1)] + a[k];
            barrier;
      }
}
```



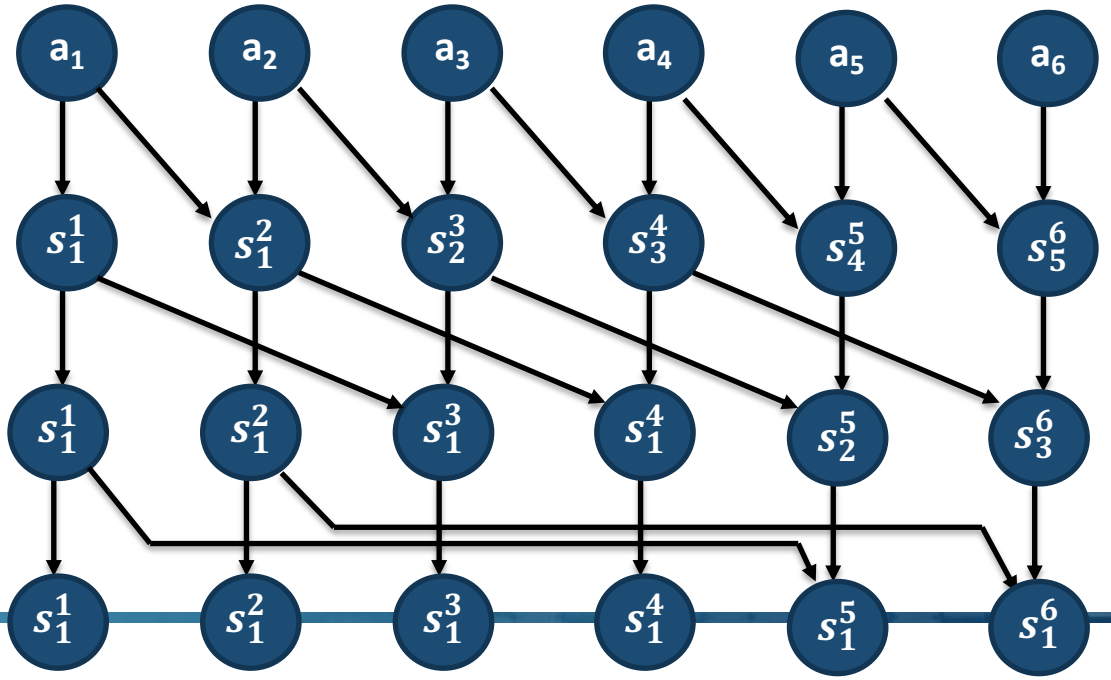
# Sume prefix (varianta 1)



# Sume prefix – varianta 1

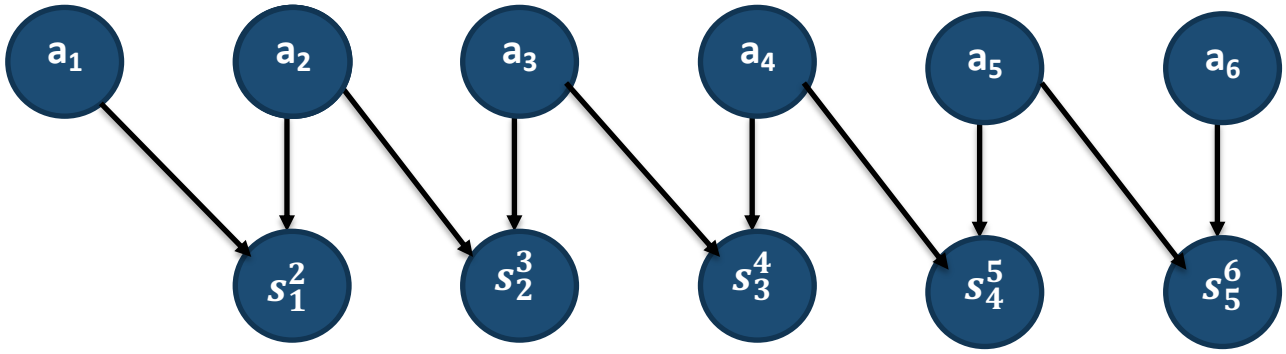
```
int a[1:n];
process suma[k=1 to n] {
      for [j = 1 to sup(log2 n)] {
            if (k - 2^(j-1) >= 1)
                  a[k] = a[k-2^(j-1)] + a[k];
            barrier;
      }
}
```

**Eroare de sincronizare...**



# Sume prefix – varianta 1 – probleme

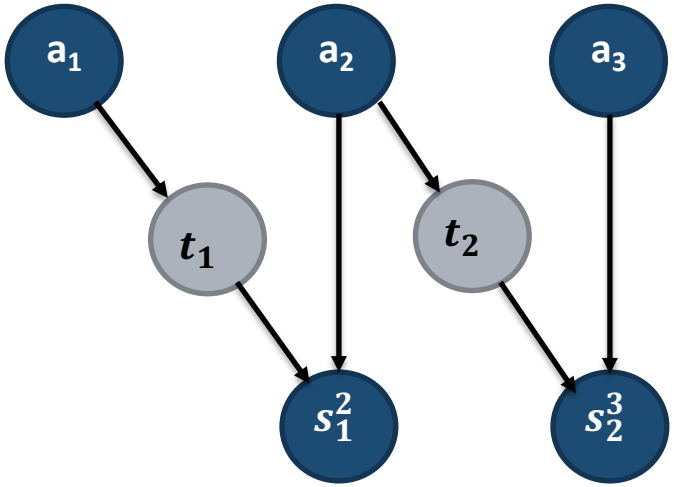
- Presupunem că procesorul numărul 3 este mai lent decât restul.
  - Suprascriere a locației de memorie




$$s_2^3 = a_3 + s_1^2$$

# Sume prefix – varianta 2

```
int a[1:n], temp[1:n];
process suma[k=1 to n] {
      for [j = 1 to sup(log2 n)] {
            temp[k] = a[k];
            barrier;
            if (k - 2^(j-1) >= 1)
                  a[k] = temp[k-2^(j-1)] + a[k];
            barrier;
      }
}
```



# Sume prefix – varianta 3

```
int a[1:n], temp[1:n];
process suma[k:1 to n] {
    int d = 1;
    while (d < n) {
      temp[k] = a[k];
      barrier;
      if (k - d >= 1) a[k] = temp[k-d] + a[k];
      barrier;
      d = 2 * d;
    }
}
```

# Parallel Scan

# Parallel Scan

scan (o) < x1, ..., xn >

==

< x1, x1 o x2, ..., x1 o ... xn >

*Exemplu: Filtru*

Considerați la **intrare** un vector, trebuie găsit la ieșire un alt **vector** care să conțină doar elementele pentru care **(f elt)** este **true**

Fie f x = x > 10

*filter f  < 17, 4, 6, 8, 11, 5, 13, 19, 0, 24 >*
*= < 17, 11, 13, 19, 24 >*

Paralelizabil?

- Găsirea elementelor este o operație ușoară
- Dar punerea lor în secvența corectă pare mai dificilă

# Prefixe paralele - to the rescue

1. Folosim mapare paralelă pentru a calcula un bit-vector pentru elementele ce se potrivesc operației

   *input:* *< 17, 4, 6, 8, 11, 5, 13, 19, 0, 24 >*
   *bits:* *< 1, 0, 0, 0, 1, 0, 1, 1, 0, 1 >*

2. Aplicăm sume prefix pe elementele bit-vectorului

   *bitsum* *< 1, 1, 1, 1, 2, 2, 3, 4, 4, 5 >*

3. Aplicăm parallel map pentru a produce ieșirea

   *output* *< 17, 11, 13, 19, 24 >*

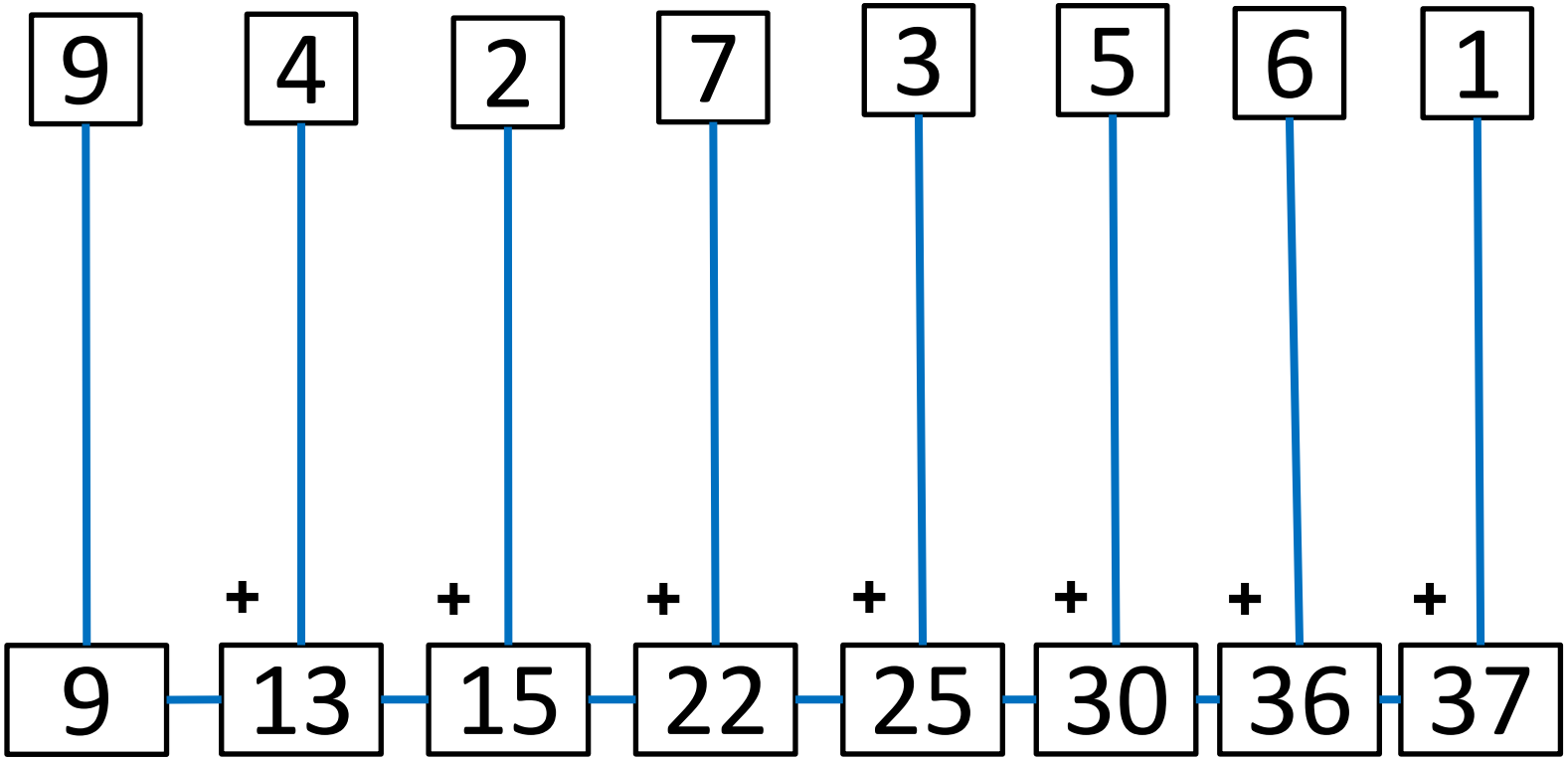
# Generic: Scan

**Se aplică aceeași operație între elementele unui vector.**
**Se colectează toate rezultatele parțiale**

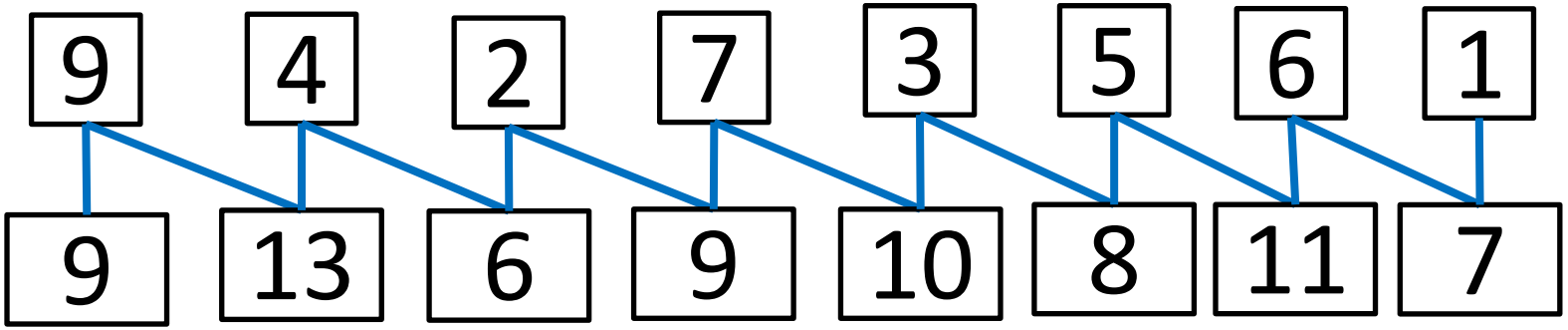
**Poate fi executat în log(n) pași folosind un arbore**

**Operația poate lua orice formă (+, *, min, max, and, etc.)**

# Scan - sum



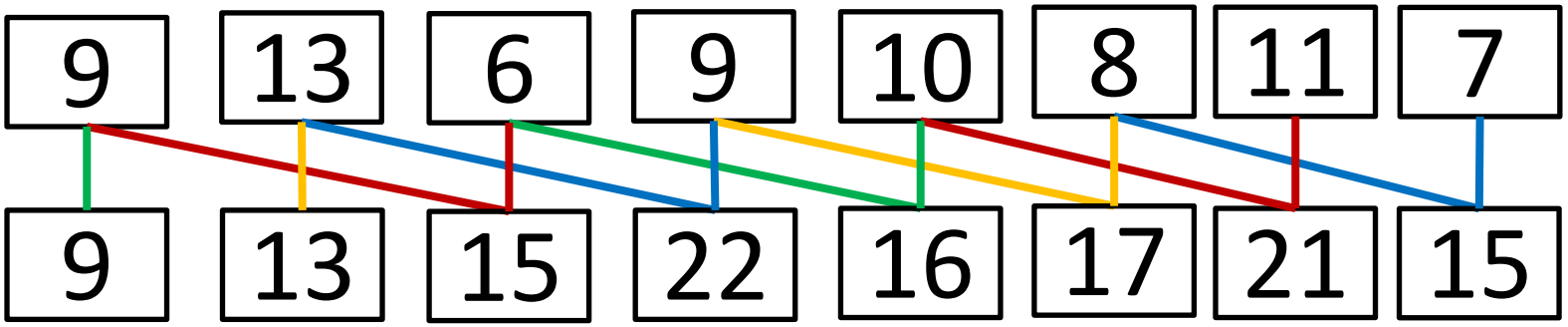
# Scan - sum




**Pot fi executate în paralel**



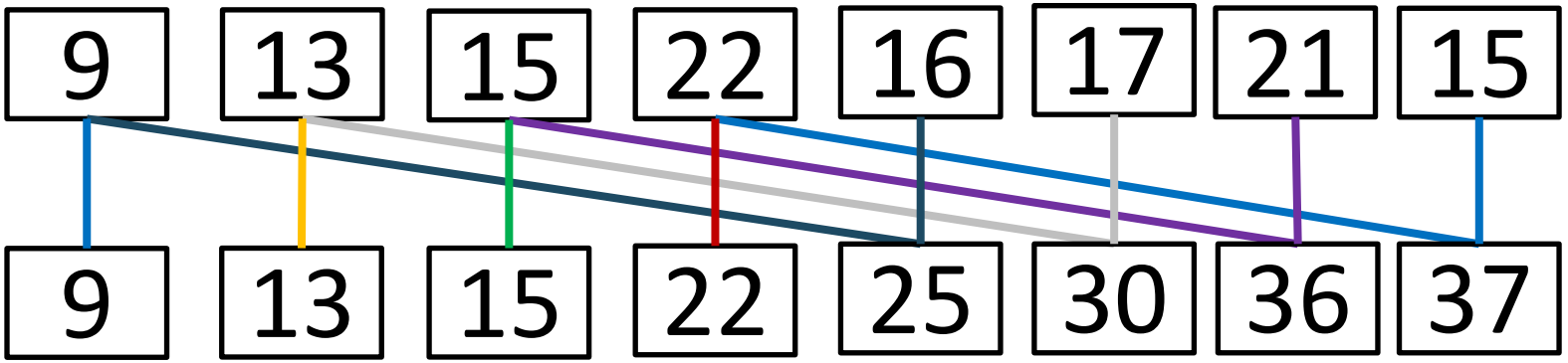
# Scan - sum




**Pot fi toate executate în paralel**

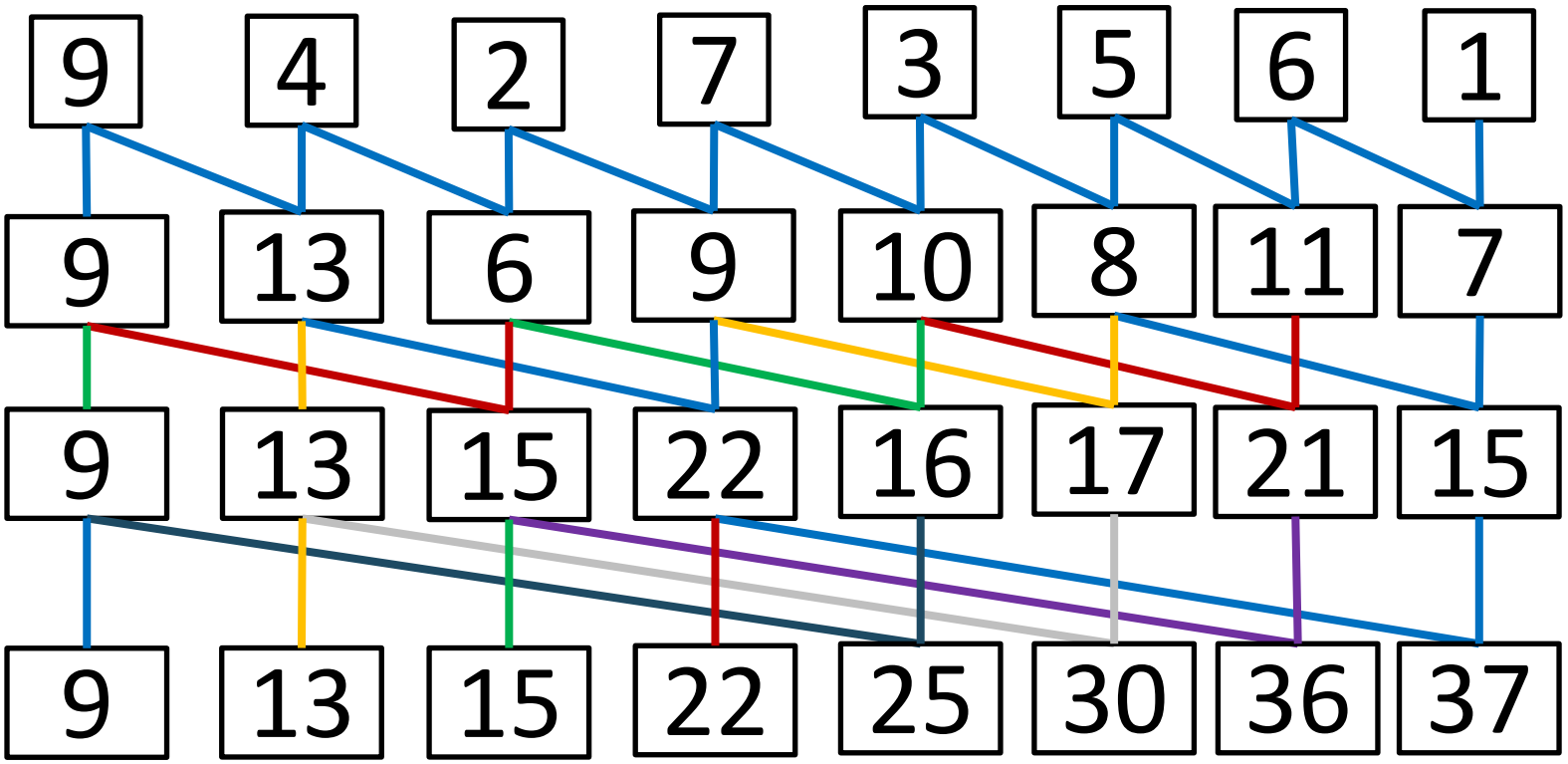


# Scan - sum

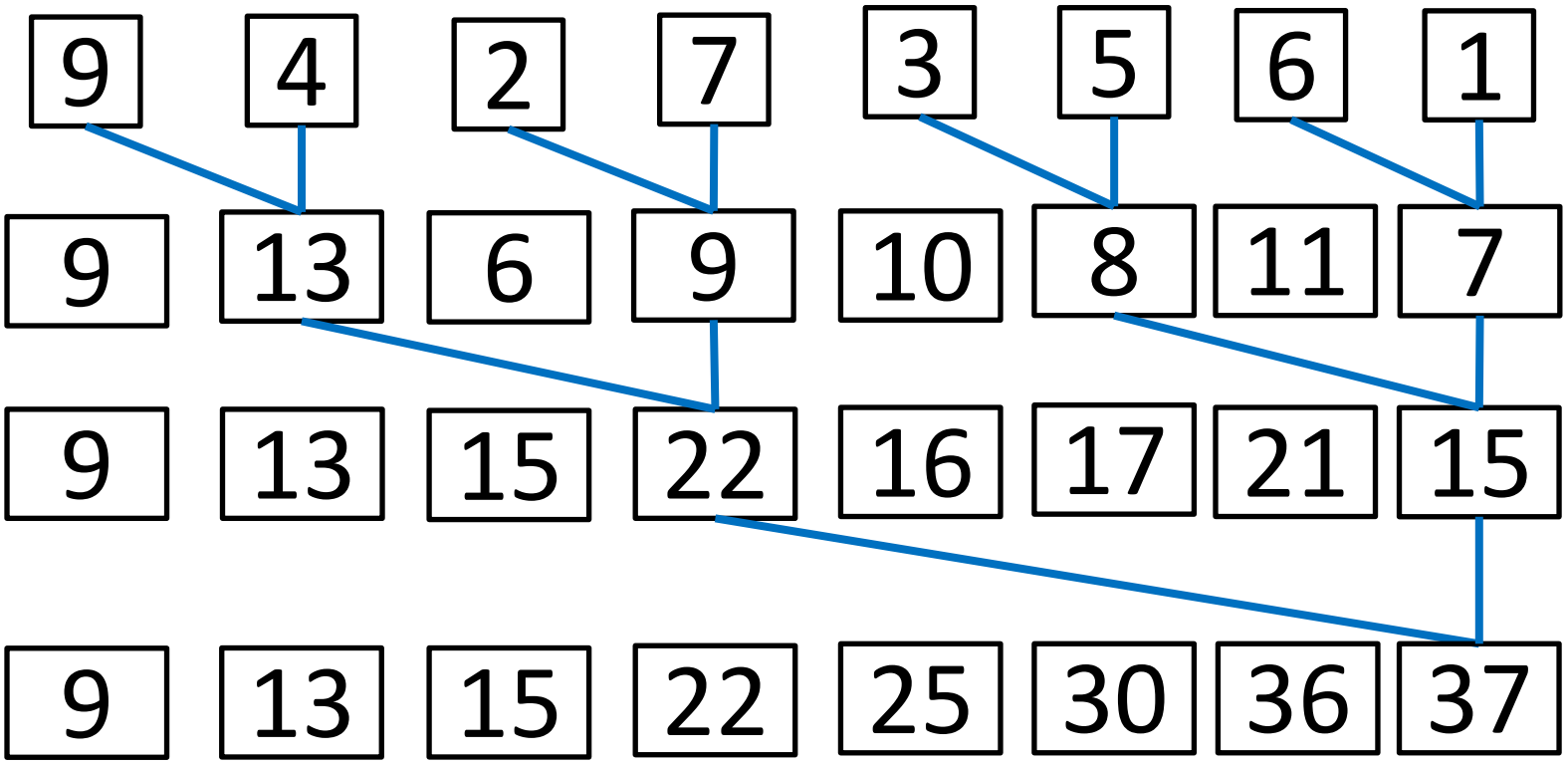



**Pot fi toate executate în paralel**

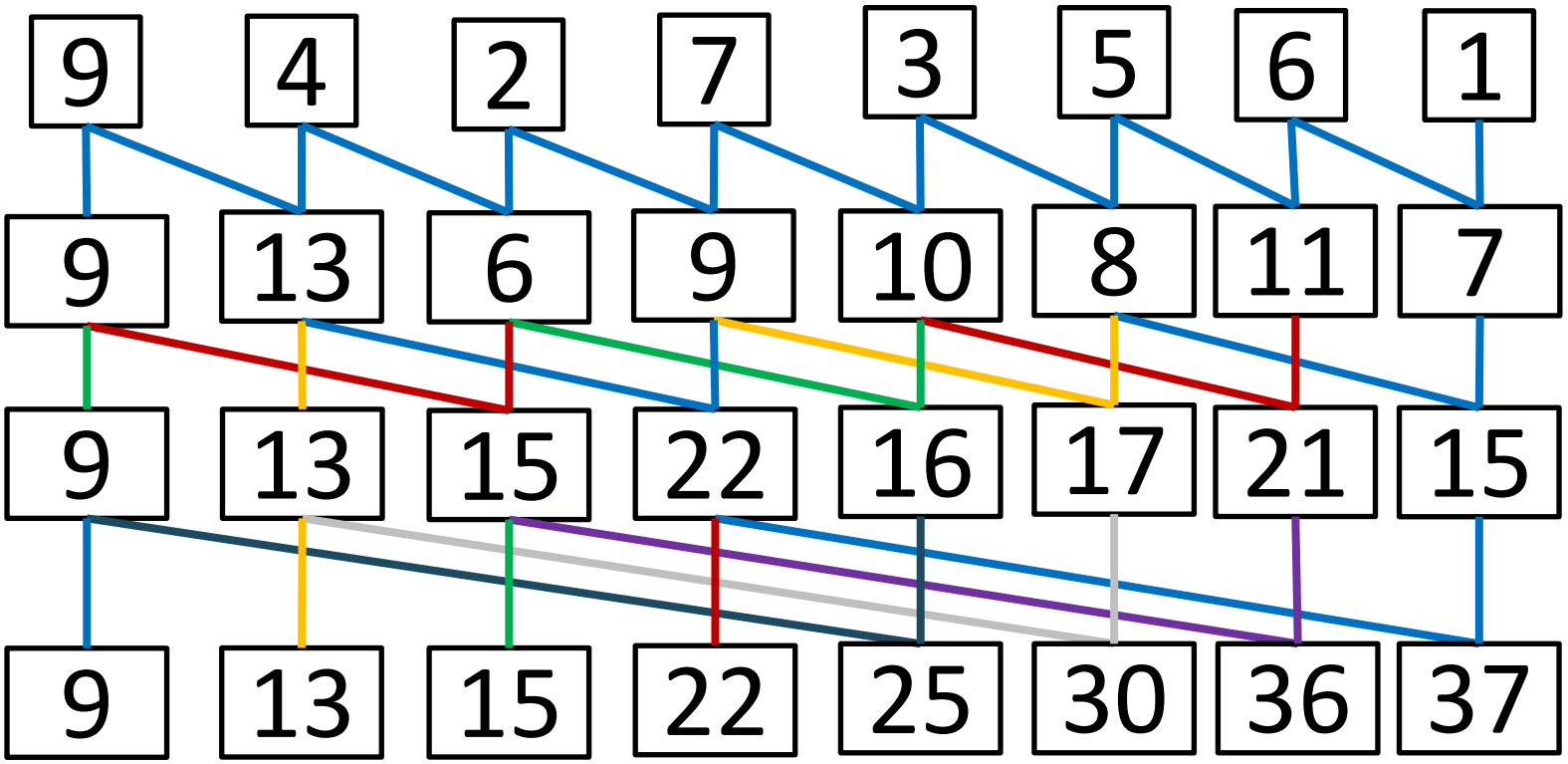
# Scan – Cum funcționează?



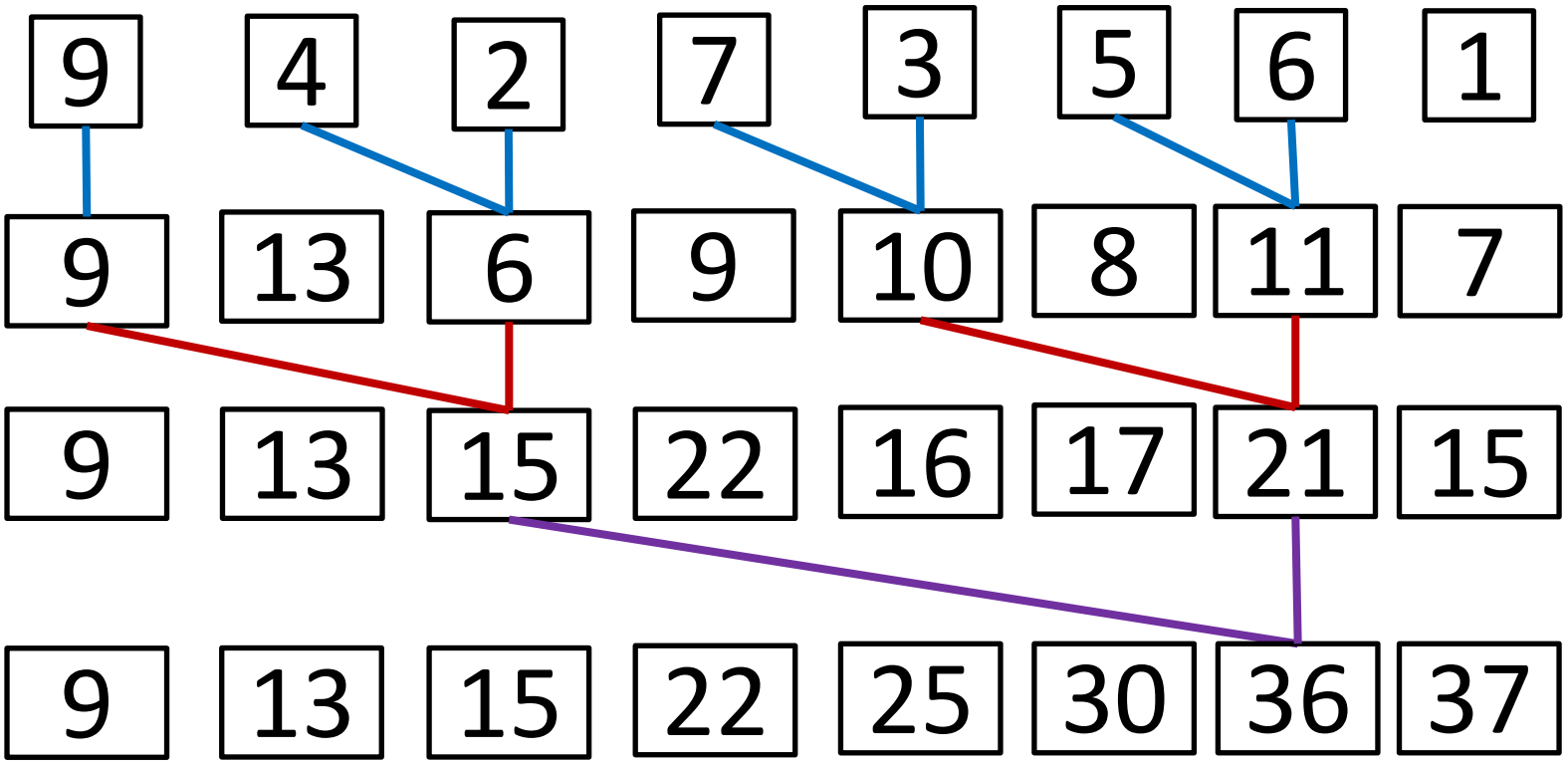
# Scan – Cum funcționează



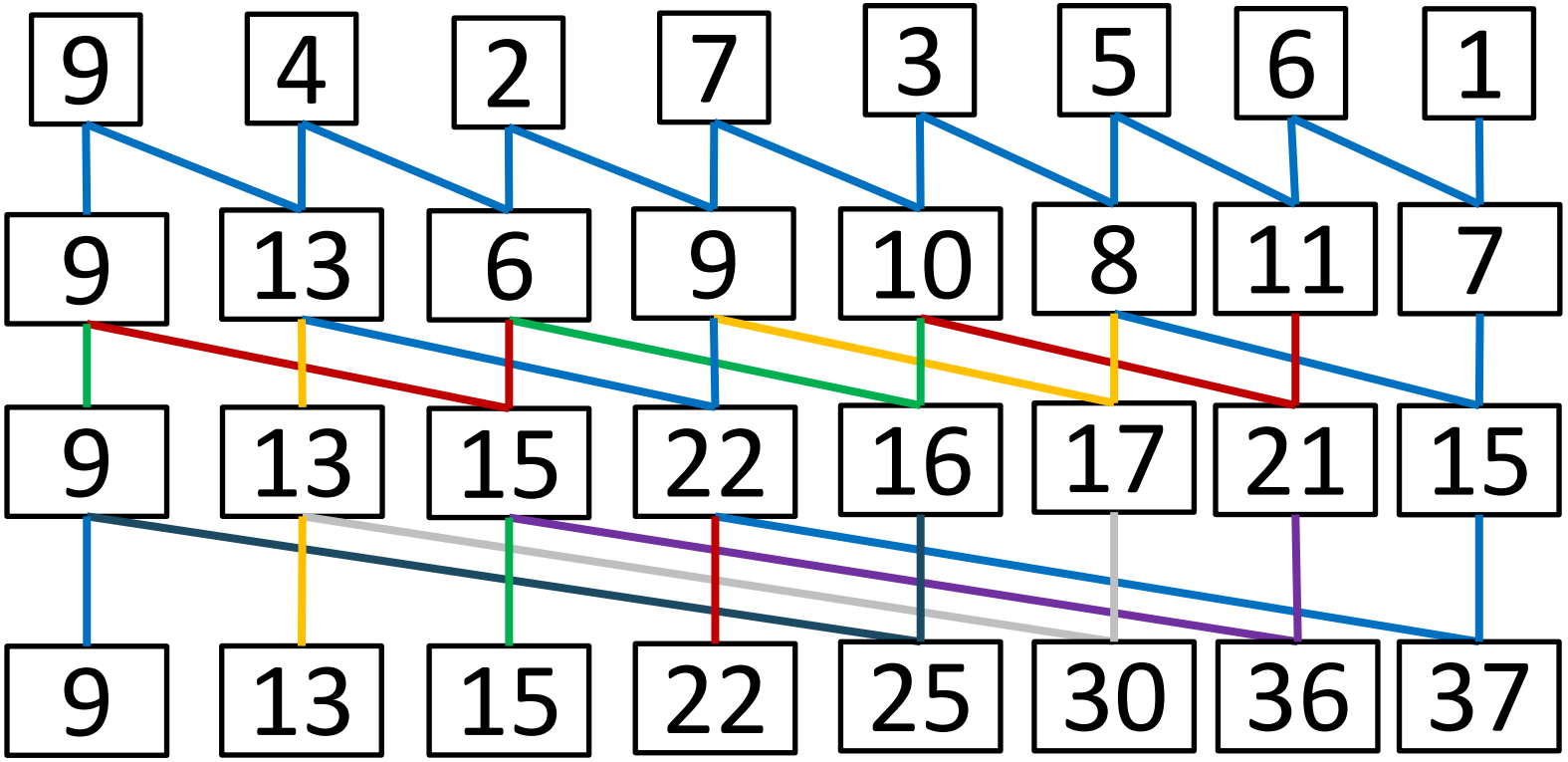
# Scan – Cum funcționează?



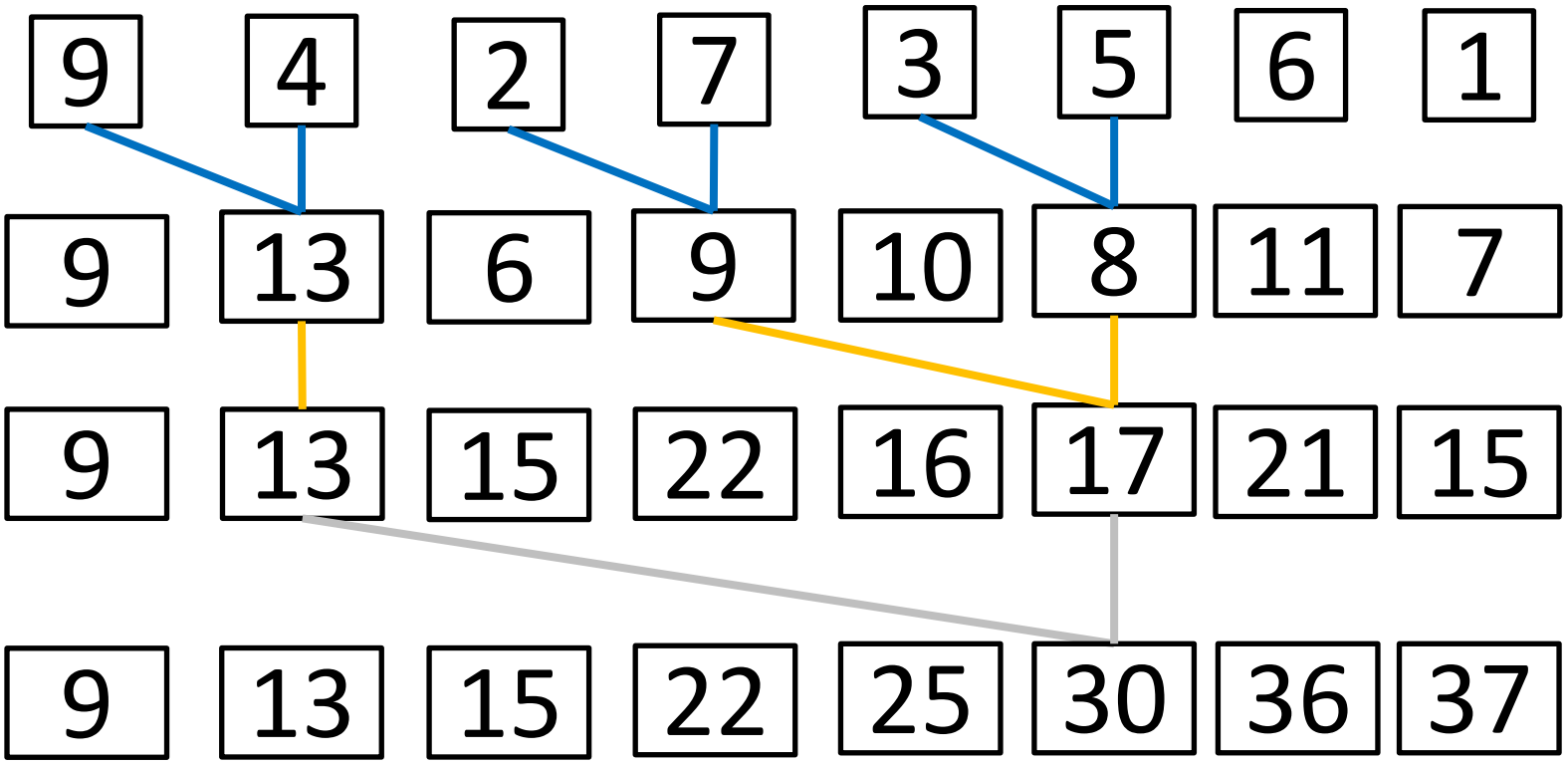
# Scan – Cum funcționează?



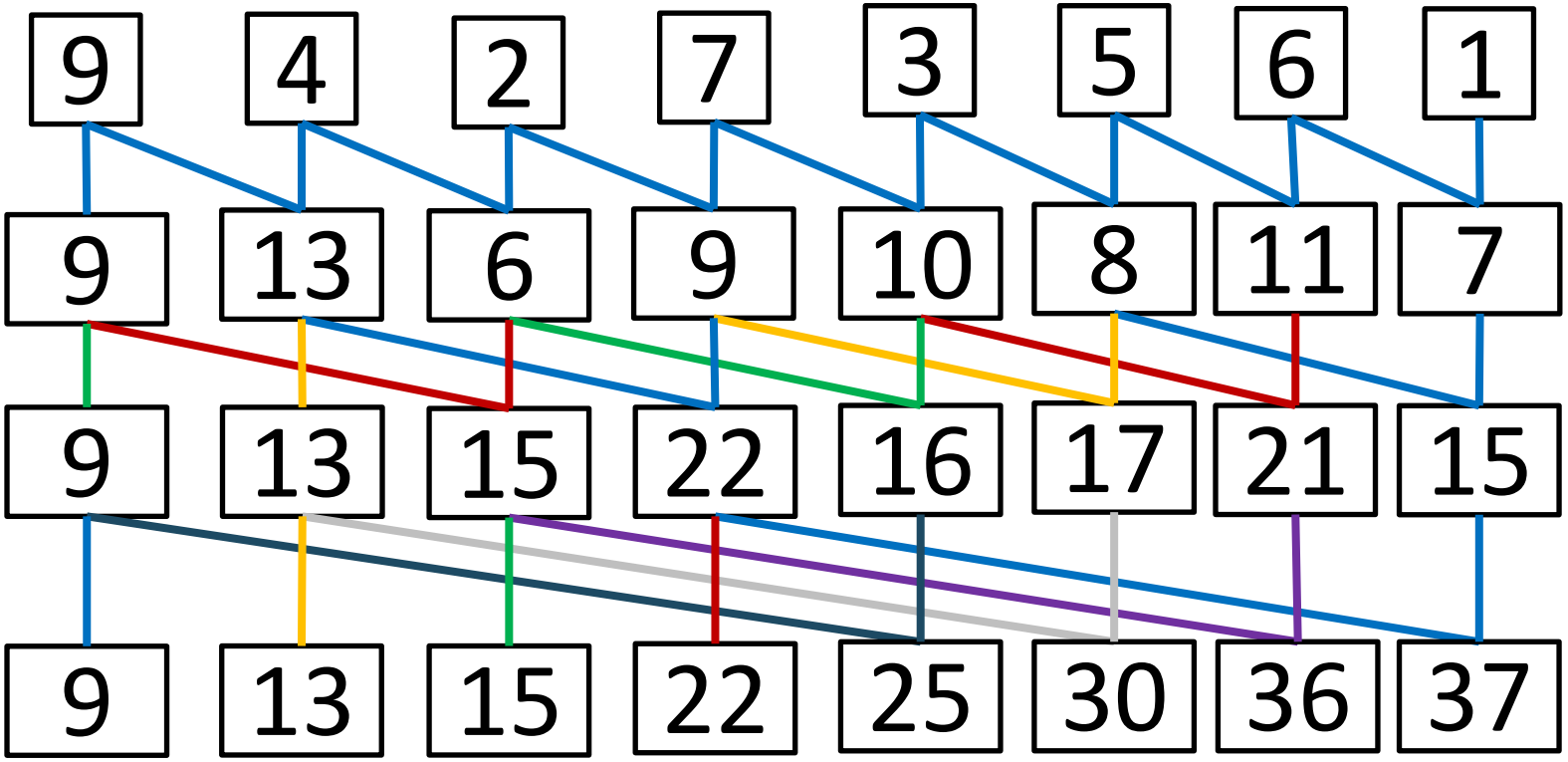
# Scan – How it works?



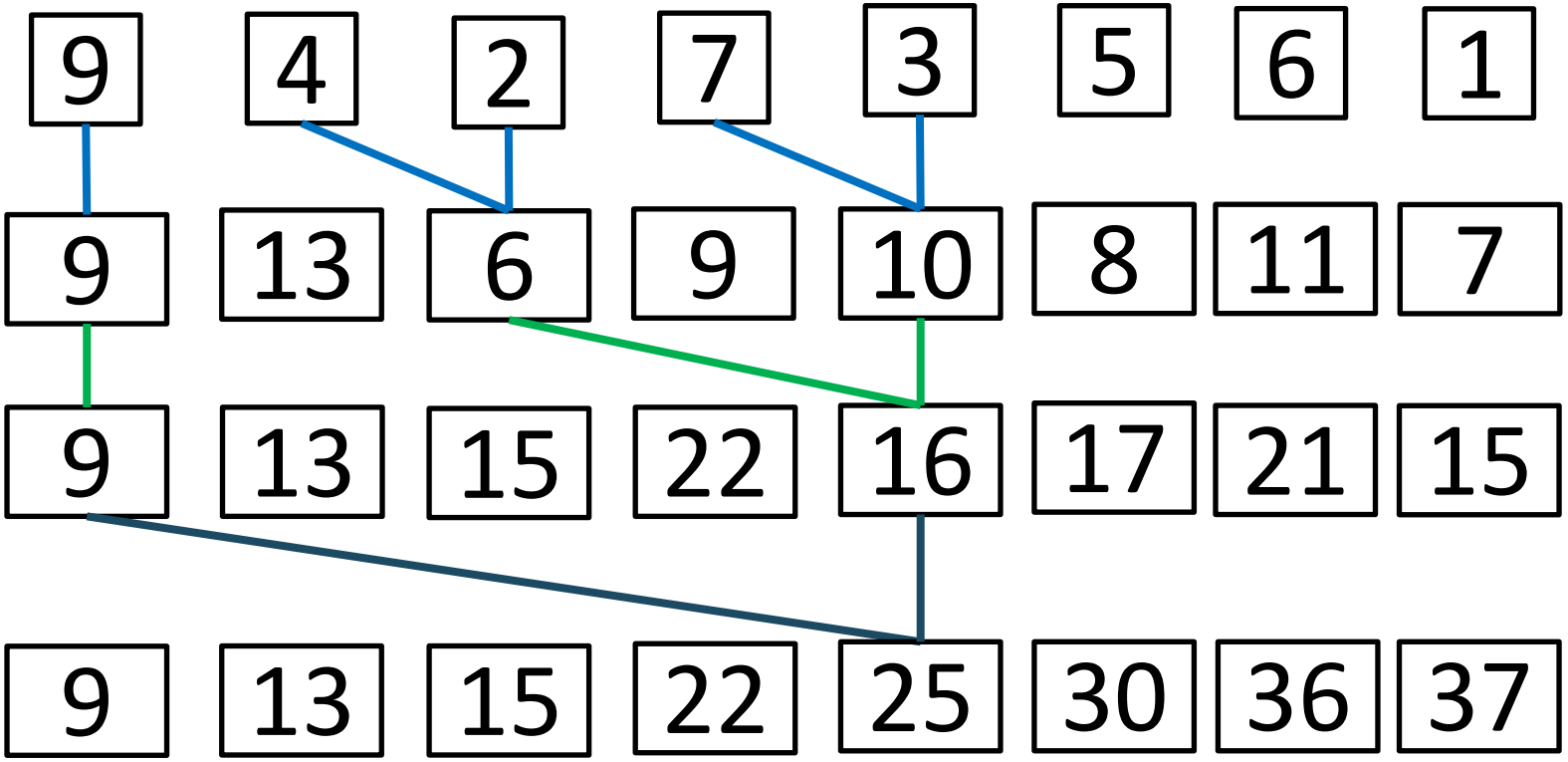
# Scan – How it works?



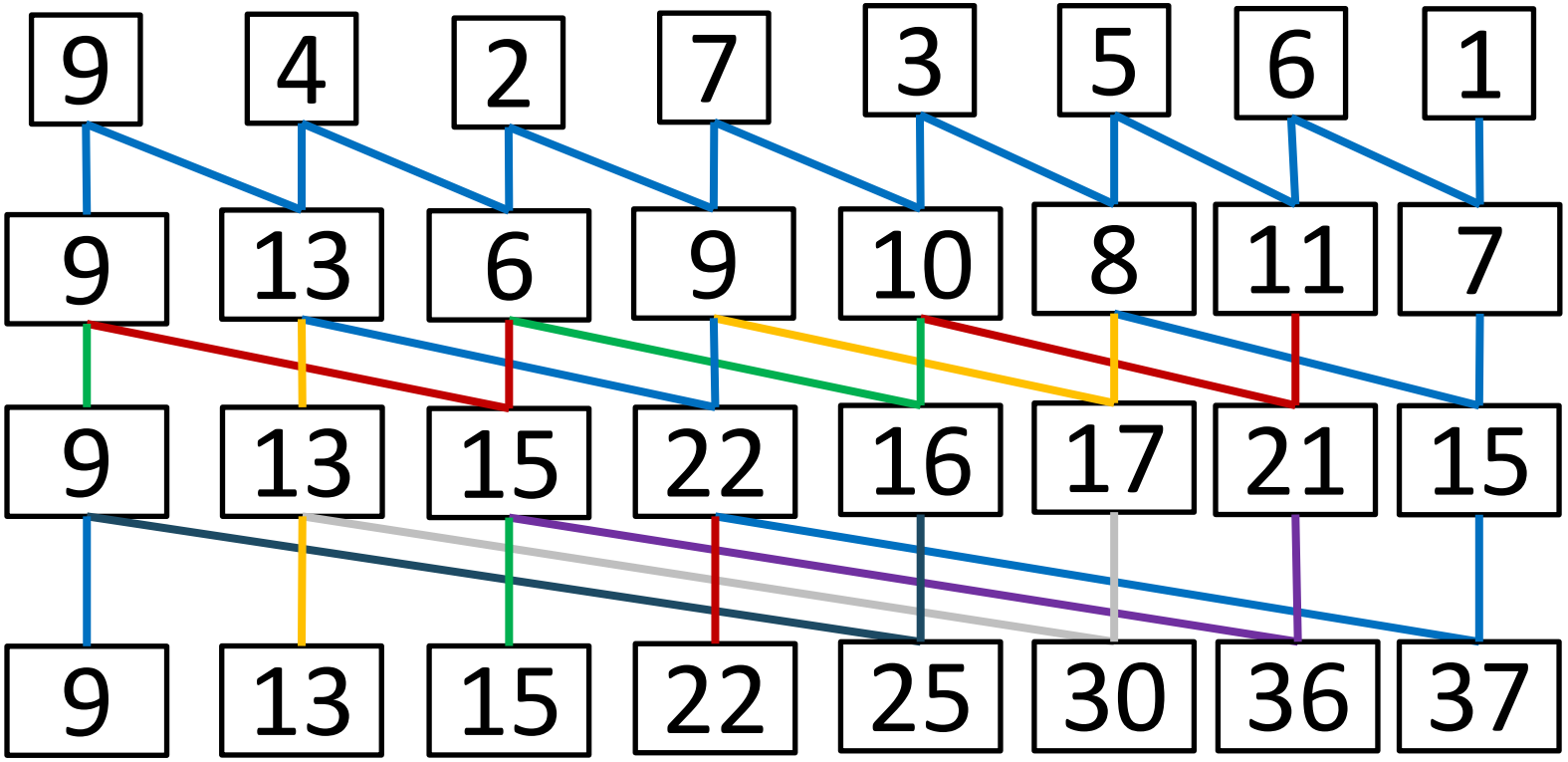
# Scan – How it works?



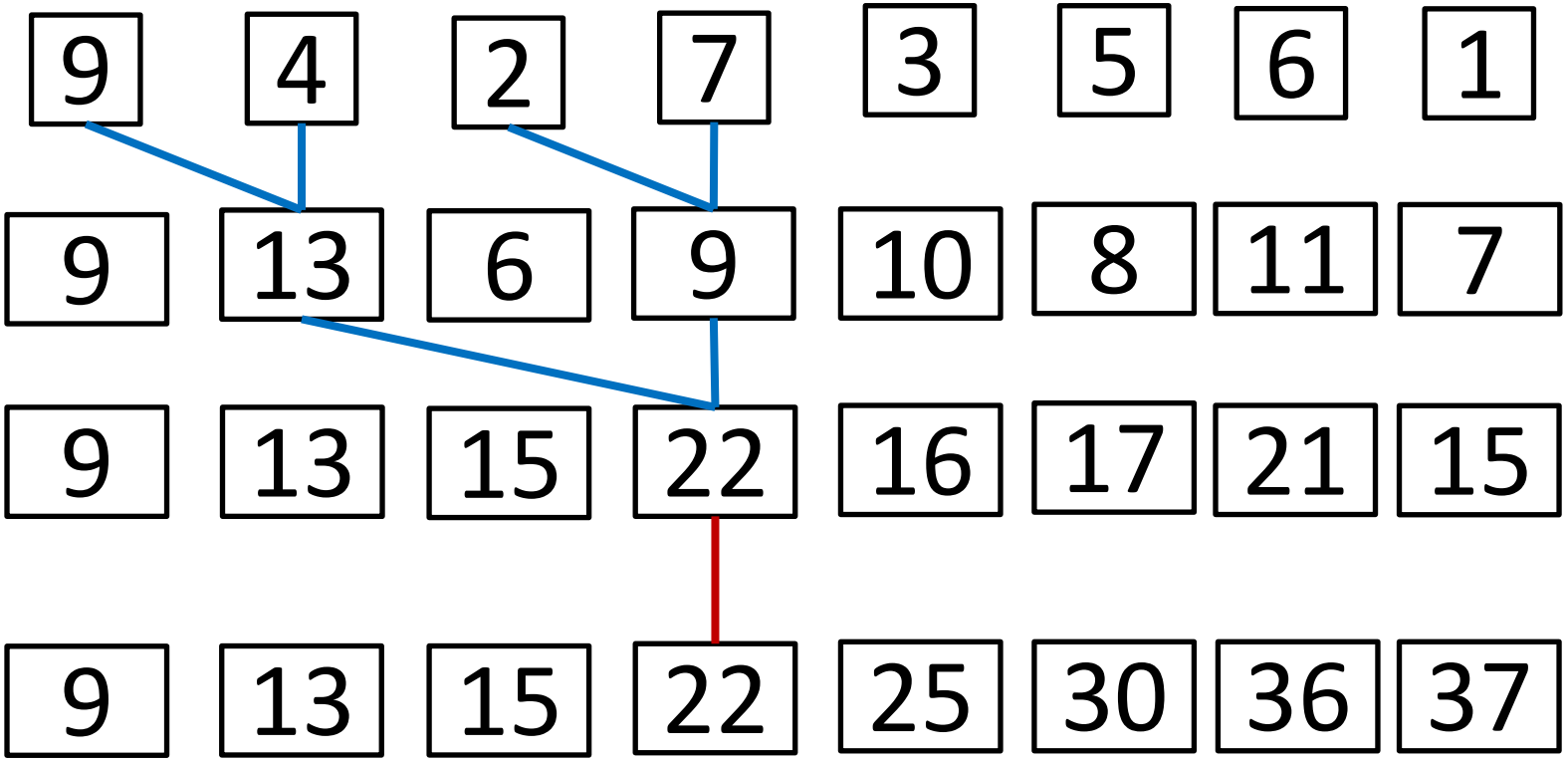
# Scan – How it works?



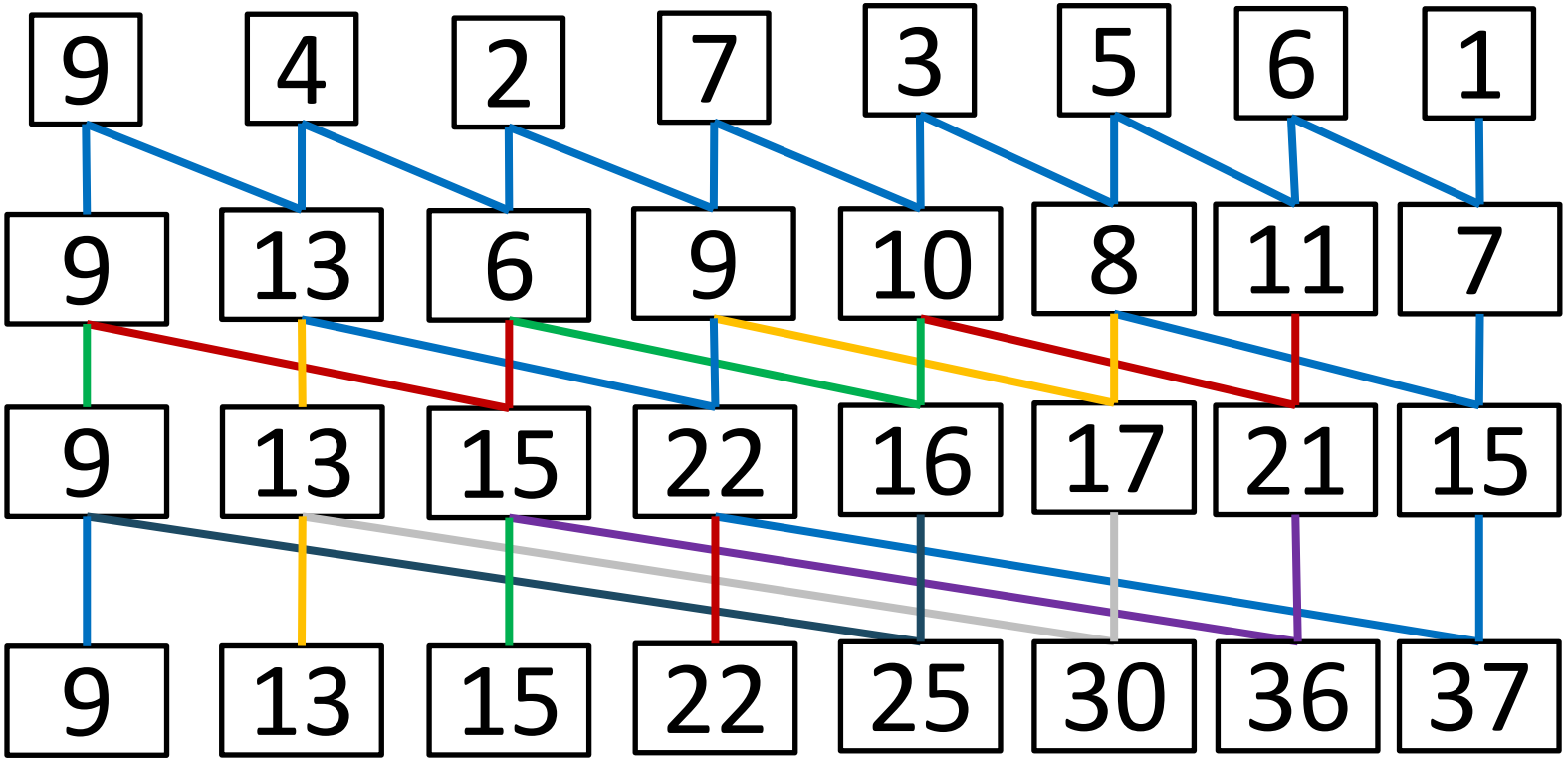
# Scan – How it works?



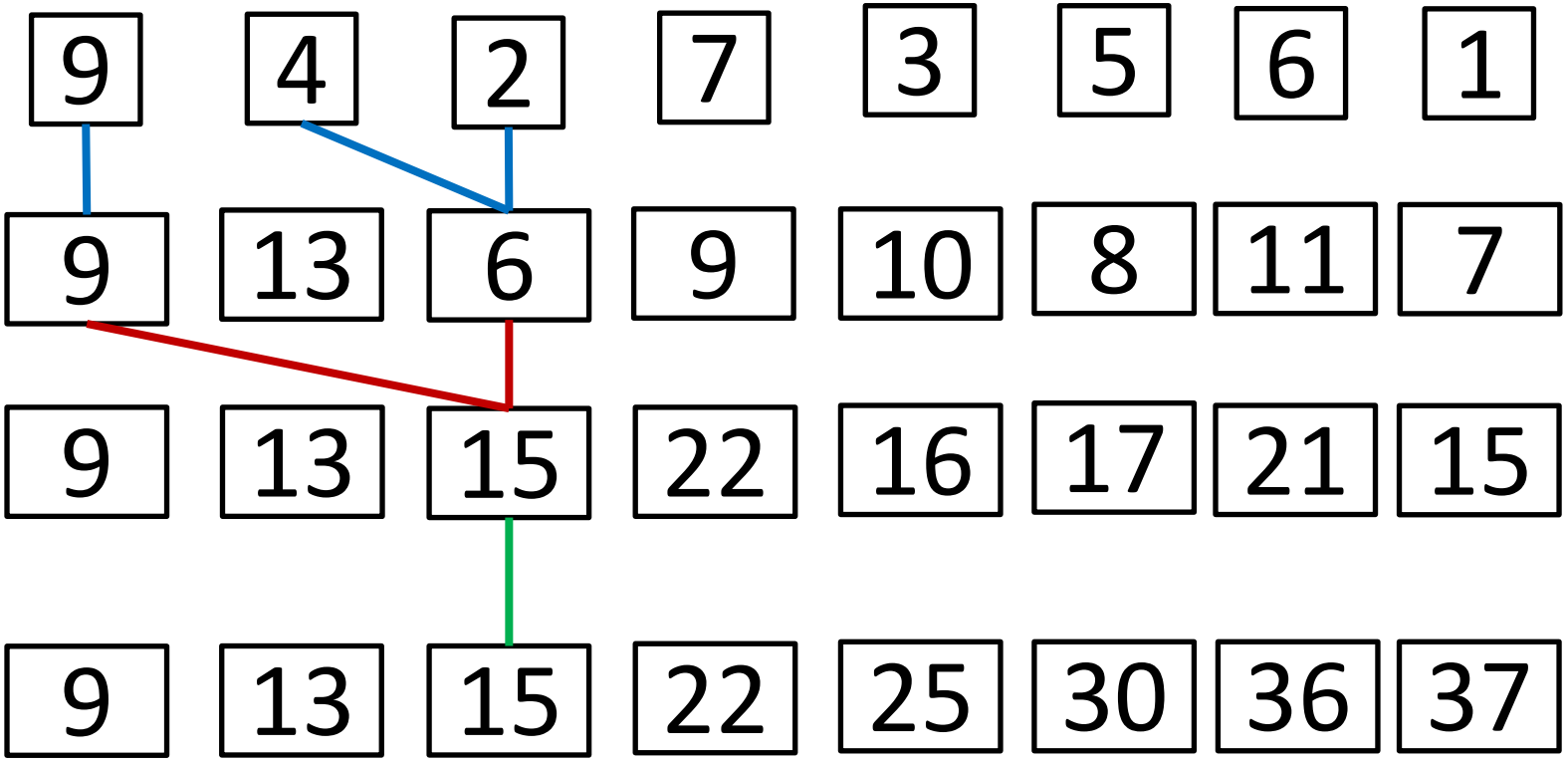
# Scan – How it works?



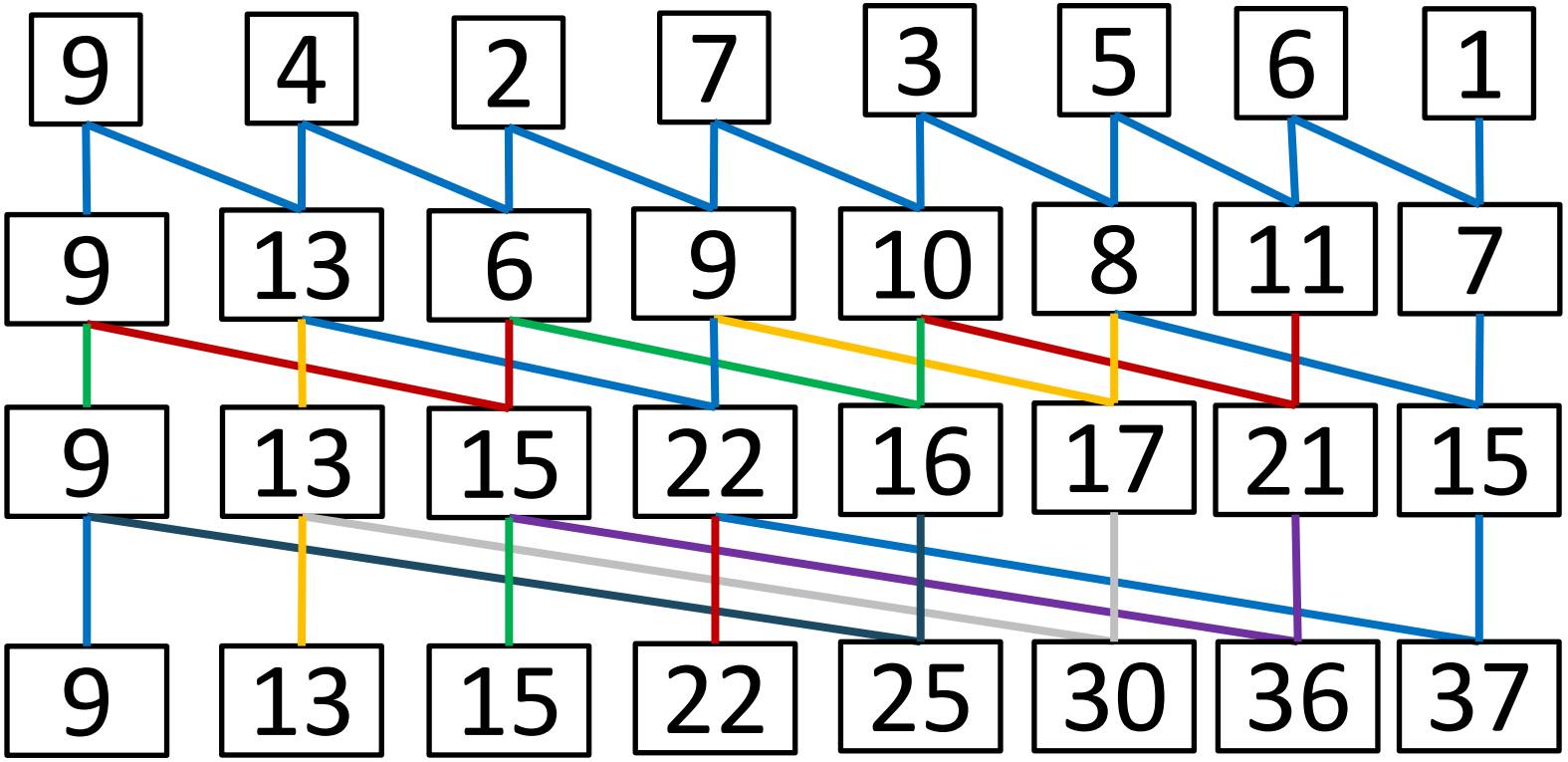
# Scan – How it works?



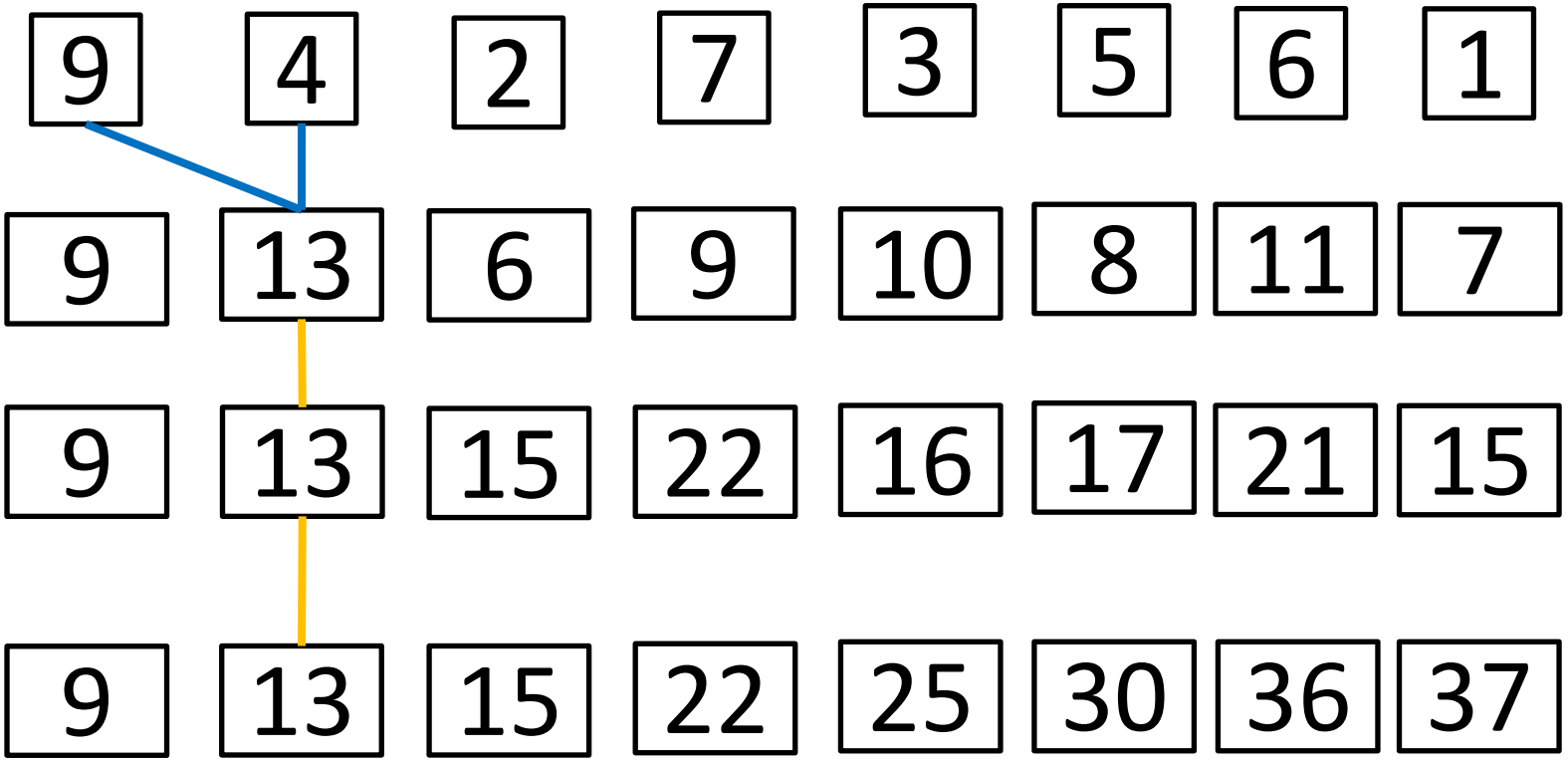
# Scan – How it works?



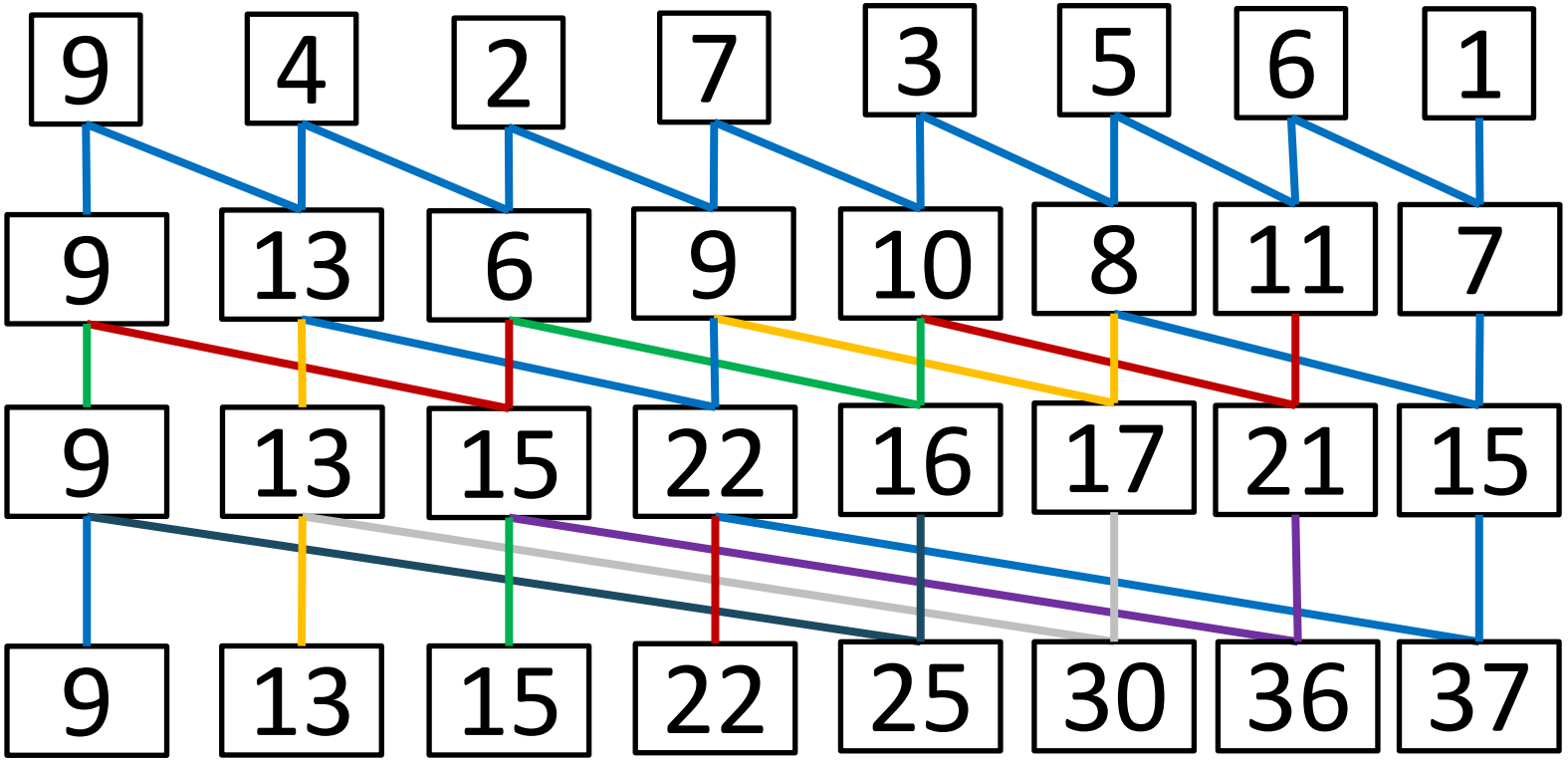
# Scan – How it works?



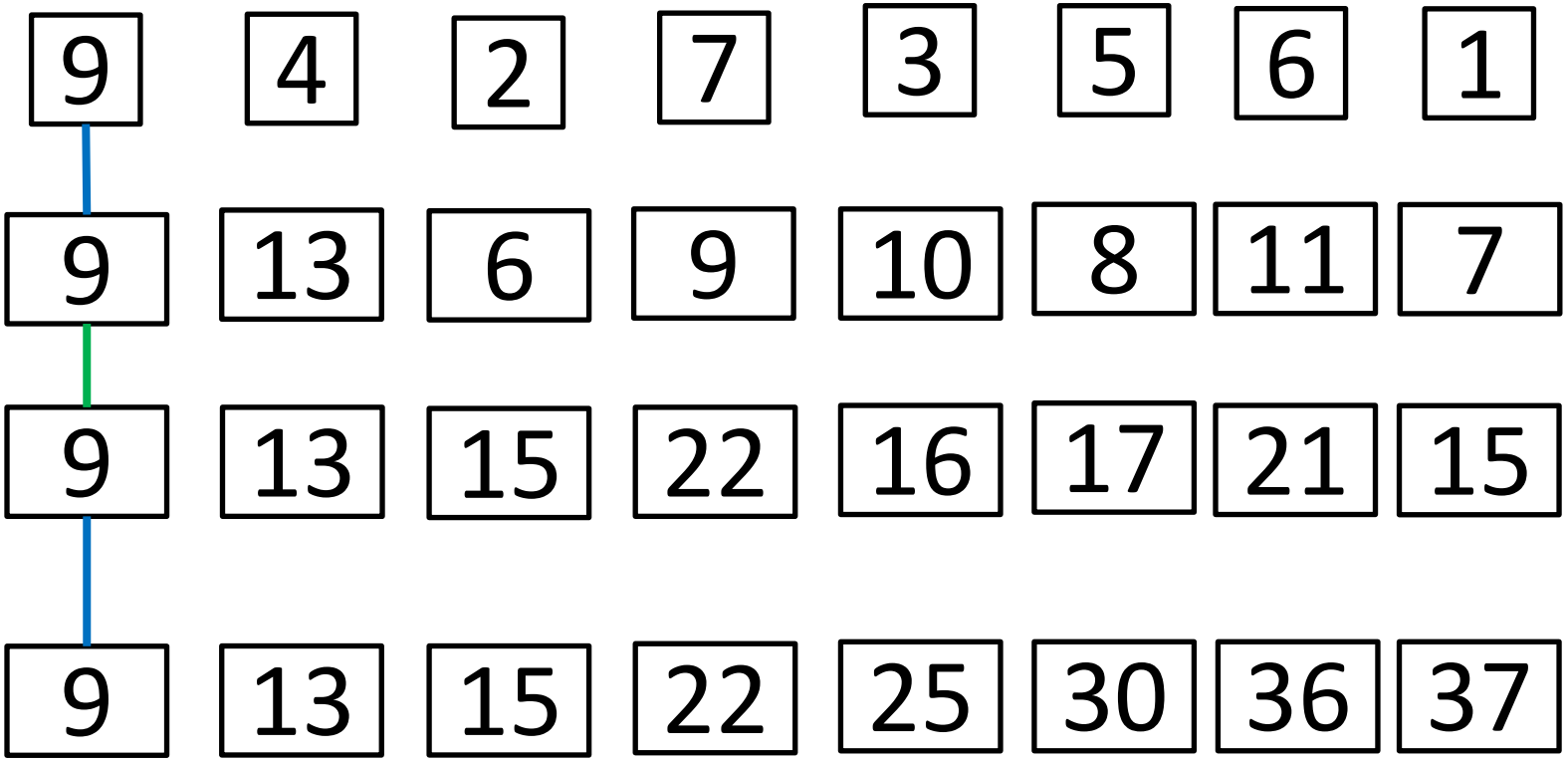
# Scan – How it works?



# Scan – How it works?



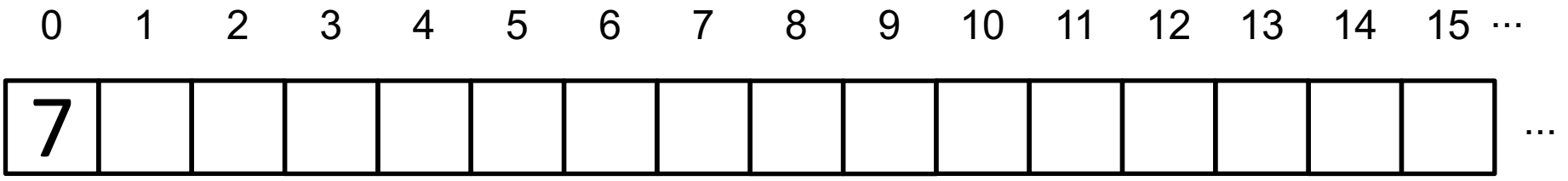
# Scan – How it works?



# Difuzarea unei valori

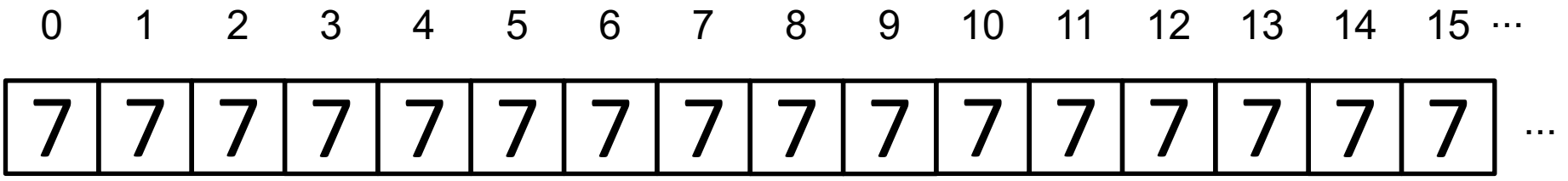
# Value Broadcast

**Start**



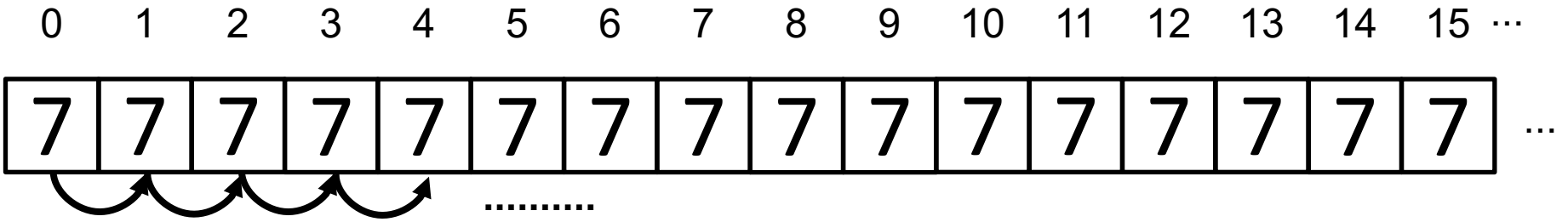
# Value Broadcast

**End**



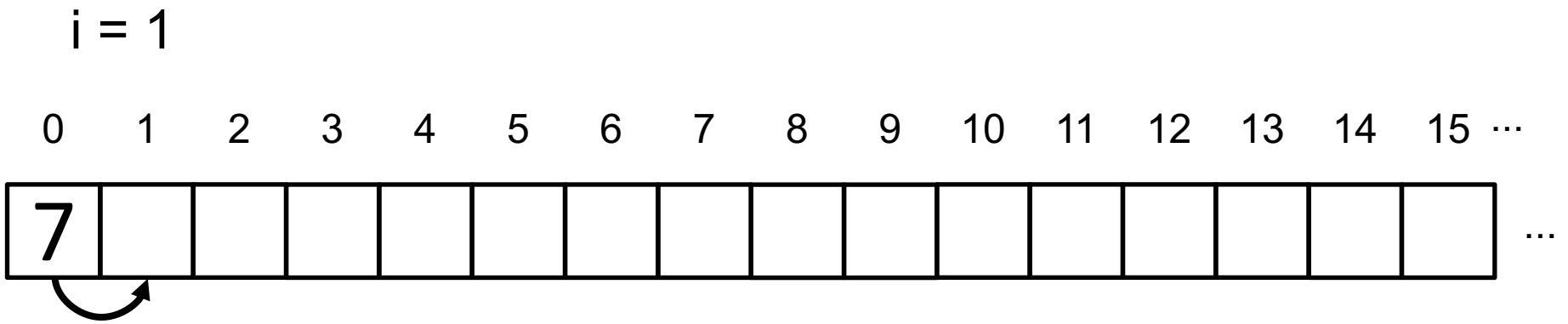
# Inefficient Value Broadcast

**O(n) time**



# Efficient Value Broadcast

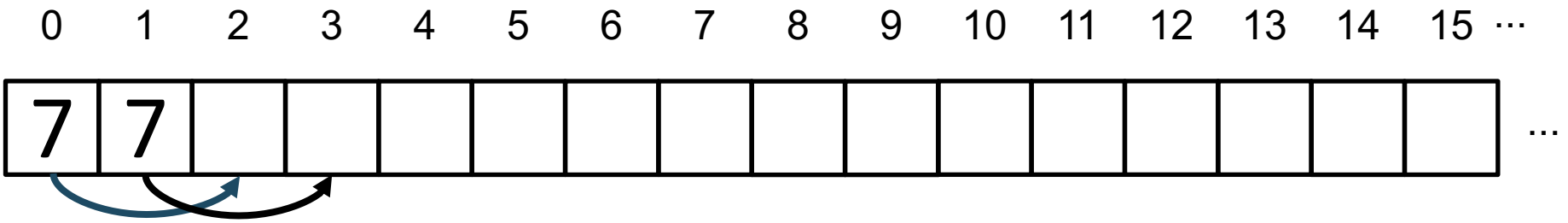
Every element that has the value
copies it to its current position + i



# Efficient Value Broadcast

Every element that has the value
copies it to its current position + i

i = 2

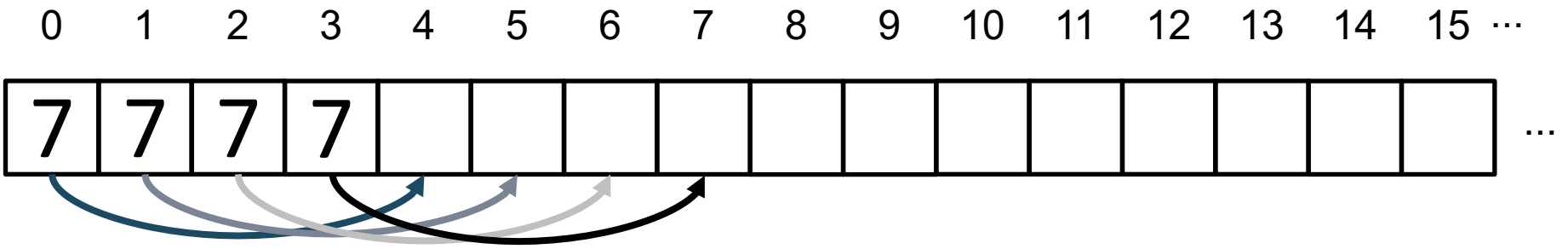



These operations can be executed in parallel

# Efficient Value Broadcast

Every element that has the value
copies it to its current position + i

i = 4

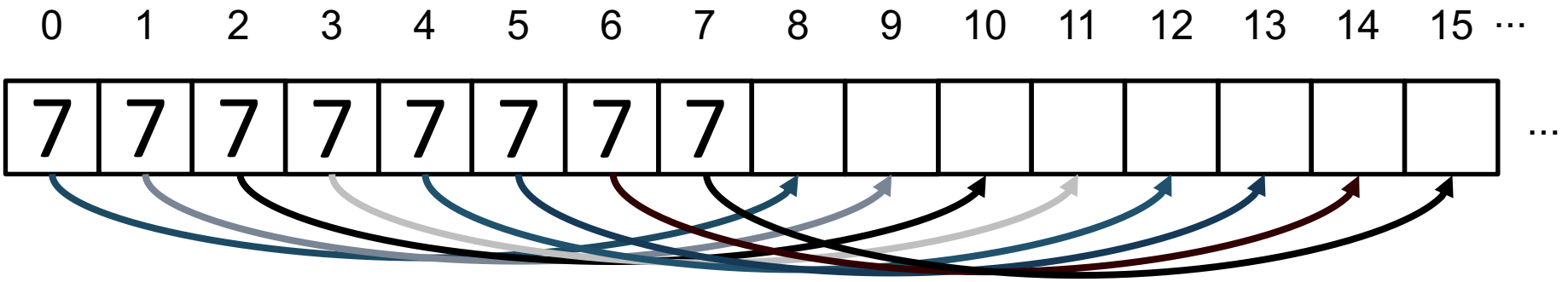



These operations can be executed in parallel

# Efficient Value Broadcast

Every element that has the value
copies it to its current position + i

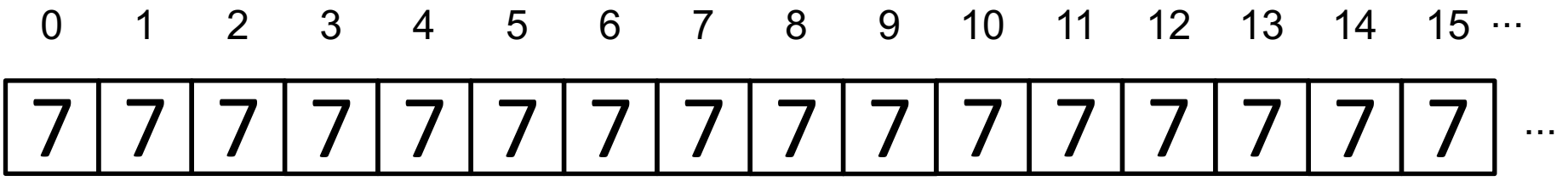
i = 8




These operations can be executed in parallel

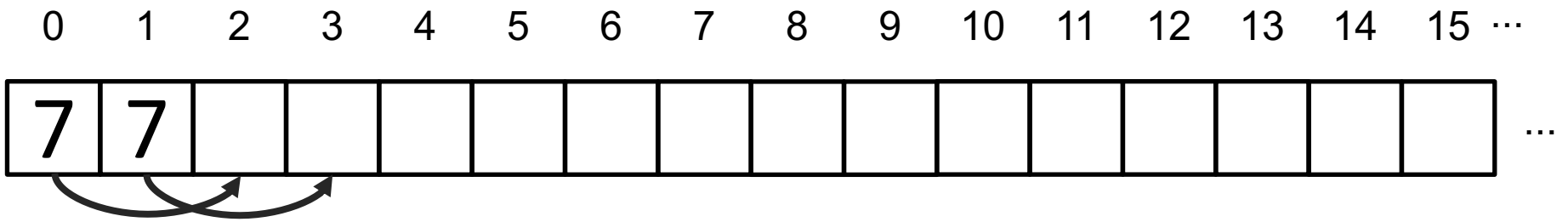
# Efficient Value Broadcast

**O(log2(n)) time with p = n/2 processors**



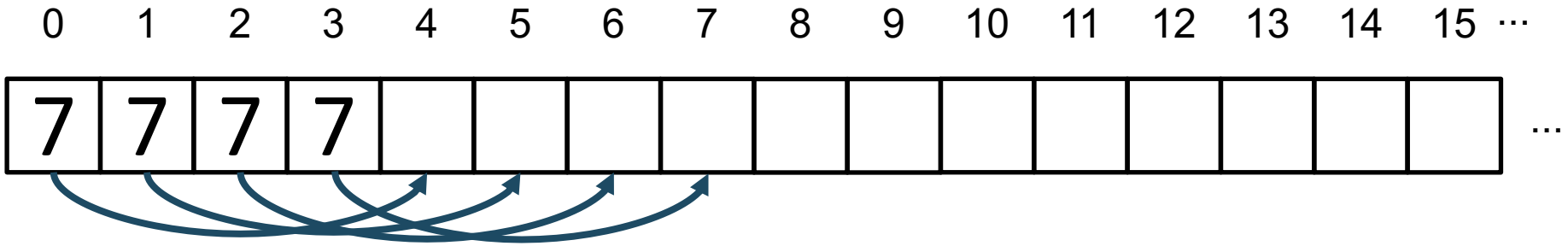
# Efficient Value Broadcast

i = 2



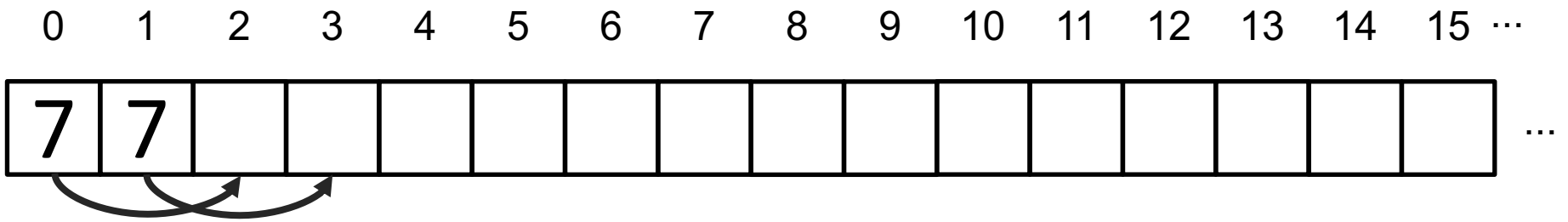

These operations can**NOT** be executed in parallel

i = 4



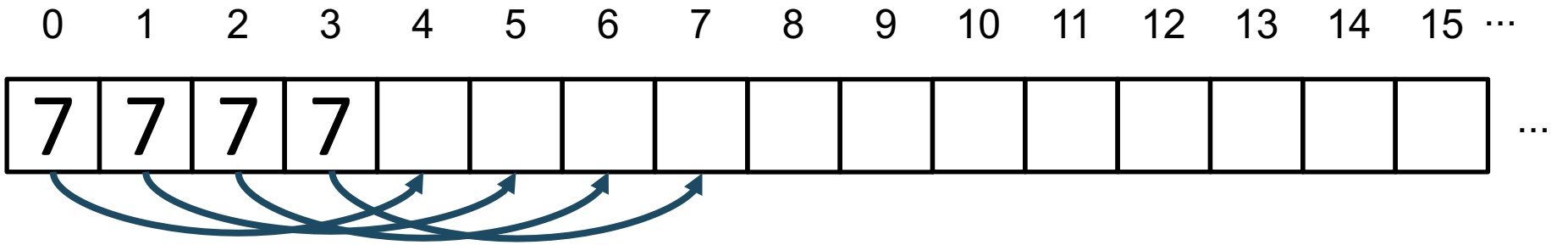
# Efficient Value Broadcast

i = 2






i = 4



# Broadcast

**Pas 1: (Procesorul $P_1$)**

```
int t;/* t locala P1 */
t = D;
A[1] = t;
```

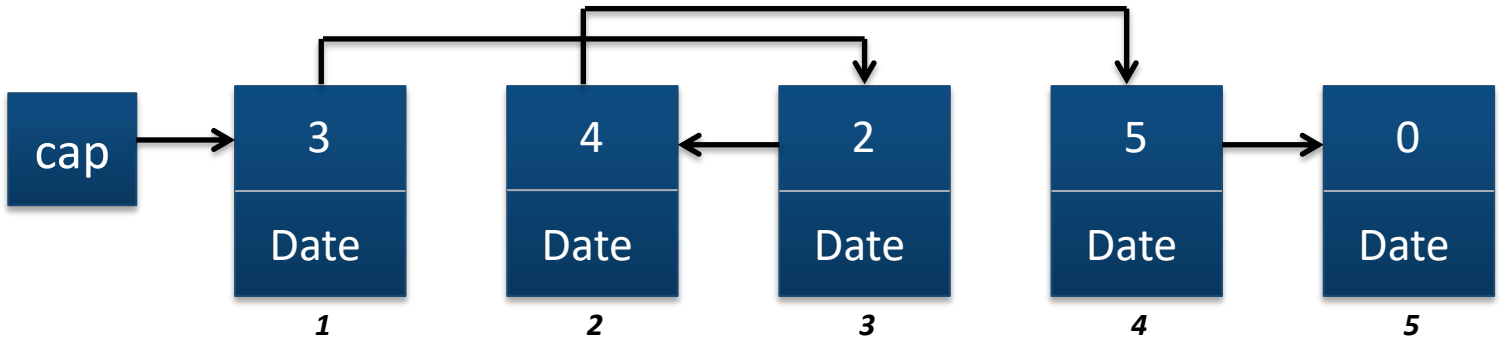
**Pas 2:**

```
for [i = 0 to (log N -1)] {
    process Pj[j = 2^i+1 to 2^i+1]{ /*(Procesor Pj)*/
        int t; /* t locală Pj */
        t = A[j-2^i];
        A[j] = t;
    }
}
```

# Operații cu liste

# Operații cu liste

- Listă *n* elemente: `data[1:n]`
- Legăturile între elemente: `leg[1:n]`
- Capul listei: `cap`
- Elementul `i` face parte din listă ⇔ fie
  - `cap = i`
  - există un element `j`, între `1` și `n`, a.î. `leg[j]=i`

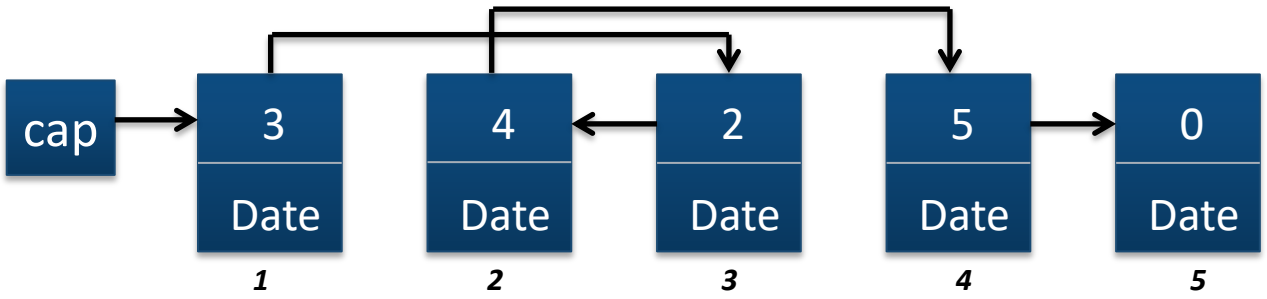



- Problemă: vrem ca fiecare procesor să afle capătul listei

# Operații cu liste

```
int leg[1:n], end[1:n];
process Află[i=1 to n] {
      int nou, d=1;
      end[i] = leg[i];
      barrier;
      while (d<n) {
            nou = 0;
            if (end[i]<>0 and end[end[i]]<>0)
                  nou=end[end[i]]
            barrier;
            if (nou<>0) end[i]=nou;
            barrier;
            d = 2*d;
      }
}
```

# Operații cu liste




| Procesor | End[i] | End[i] | End[i] | End[i] |
|---|---|---|---|---|
| 1 | 3 | 2 | 5 | 5 |
| 2 | 4 | 5 | 5 | 5 |
| 3 | 2 | 4 | 5 | 5 |
| 4 | 5 | 5 | 5 | 5 |
| 5 | 0 | 0 | 0 | 0 |
| | *init* | *d = 1*<br>*d < 5* | *d = 2*<br>*d < 5* | *d = 4*<br>*d < 5* |

```
int leg, end[1:n];
process Află[i=1 to n] {
    int nou, d=1;
    end[i] = leg[i];
    barrier;
    while (d < n) {
        nou = 0;
        if (end[i]<>0 and end[end[i]]<>0)
            nou = end[end[i]]
        barrier;
        if (nou <> 0)
            end[i] = nou;
        barrier;
        d = 2 * d;
    od
oc
```